BIT404E-OBJECT ORIENTED ANALYSIS AND DESIGN LAB

Faculty: Dr.S.Sree Dharinya

Project Title: *Café Coffeeto – Smart Café Management System*

NAME : Nitheeshwar S

Reg NO : 23BIT0302

Date : 12/01/2026

I Problem Statement

Traditional café operations involve manual handling of menu updates, order taking, inventory tracking, billing, and delivery coordination. This manual process leads to inefficiencies, errors in order management, stock shortages, delays in service, and poor customer experience.

The goal of this project is to design and develop a **Smart Café Management System** that automates **menu display, order processing, inventory tracking, payment handling, and delivery status tracking** through a web-based application. The system improves operational efficiency, reduces human error, and enhances customer satisfaction.

II Software Requirements Specification (SRS)

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements of the **Café Coffeeto – Smart Café Management System**. The purpose of this system is to digitalize and automate café operations including menu management, inventory control, order processing, delivery management, and payment handling.

This document is intended for:

- Project evaluators and faculty
- Developers
- System designers
- Stakeholders of Café Coffeeto

1.2 Scope

The **Café Coffeeto Smart Café Management System** is a web-based application designed to manage end-to-end café operations. The system provides:

- A digital menu for customers
- Online order placement and tracking
- Inventory management for café staff
- Payment processing
- Delivery/order status tracking
- Admin dashboard for monitoring operations

The system aims to improve operational efficiency, reduce manual errors, and enhance customer experience.

1.3 Definitions, Acronyms, and Abbreviations

Term	Description
SRS	Software Requirements Specification
Admin	Café manager or authorized staff
User	Customer using the system
Inventory	Stock of ingredients and products
UI	User Interface
API	Application Programming Interface

1.4 References

- IEEE 830 / IEEE 29148 SRS Standards
- Web Application Development Documentation

---

2. Overall Description

2.1 Product Perspective

The system is a standalone web-based application that can be accessed through modern web browsers. It may integrate with third-party services such as payment gateways and delivery tracking services.

2.2 Product Functions

The major functions of the system include:

- Displaying café menu categories and items
- Managing inventory stock levels
- Placing and managing customer orders
- Processing online payments

- Managing delivery or pickup orders
- Generating reports for admin

2.3 User Classes and Characteristics

User Class	Description
Customer	Views menu, places orders, makes payments
Admin	Manages menu, inventory, orders, and reports
Staff	Views orders and updates order status

2.4 Operating Environment

- Frontend: HTML, CSS, JavaScript
- Backend: Node.js, Express.js
- Database: MongoDB
- Platform: Web browser (Chrome, Firefox, Edge)
- OS: Windows, macOS, Linux

2.5 Design and Implementation Constraints

- Internet connectivity required
- Secure payment gateway integration
- Compliance with basic web security standards

2.6 Assumptions and Dependencies

- Users have access to internet-enabled devices
- Payment gateway services are available
- Admin credentials are managed securely

---

3. Specific Requirements

3.1 Functional Requirements

3.1.1 User Registration and Login

- The system shall allow users to register and log in
- The system shall authenticate users securely

3.1.2 Menu Management

- The system shall display menu items with categories
- The admin shall be able to add, update, or delete menu items

3.1.3 Order Management

- The system shall allow customers to place orders
- The system shall generate unique order IDs
- Staff shall update order status (Preparing, Ready, Delivered)

3.1.4 Inventory Management

- The system shall track inventory levels
- The system shall notify admin when stock is low

3.1.5 Payment System

- The system shall support online payments
- The system shall generate payment confirmations

3.1.6 Delivery Management

- The system shall manage delivery or pickup options
- Customers shall track order status

3.2 Non-Functional Requirements

3.2.1 Performance

- The system shall respond to user actions within 2 seconds

3.2.2 Security

- User data shall be encrypted
- Secure authentication mechanisms shall be used

3.2.3 Usability

- The system shall be easy to use for non-technical users
- The UI shall be responsive

3.2.4 Reliability

- The system shall ensure 99% uptime

---

4. External Interface Requirements

4.1 User Interfaces

- Web-based responsive UI
- Admin dashboard interface

4.2 Hardware Interfaces

- Works on standard desktops, laptops, and smartphones

4.3 Software Interfaces

- Payment Gateway APIs
- Database Management System

---

5. System Features (Use Case Summary)

Feature	Description
Digital Menu	View menu items and categories
Order Placement	Place and manage orders
Inventory Tracking	Track stock levels
Payments	Online payment processing
Delivery Tracking	Track order status

---

6. Future Enhancements

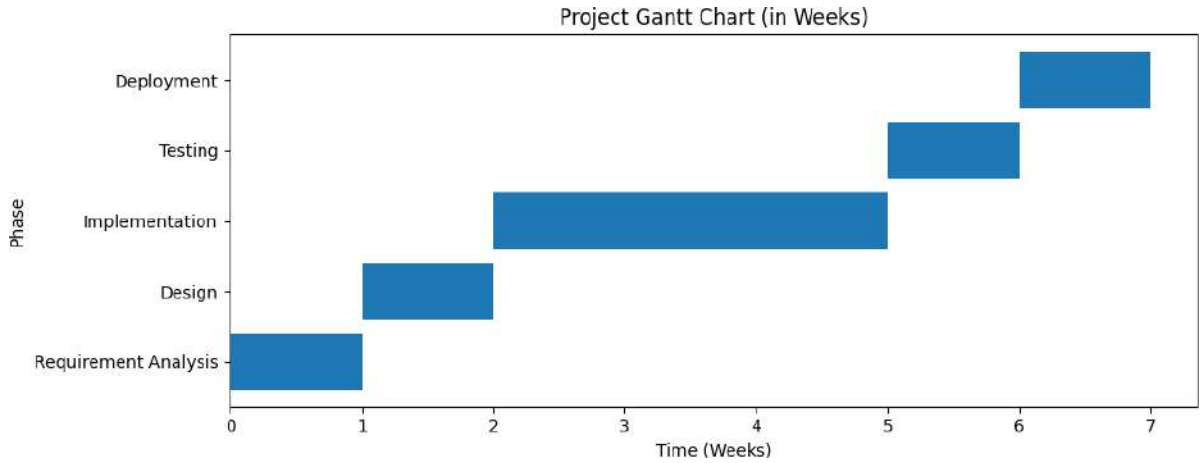
- AI-based demand prediction
- Loyalty and reward system
- Mobile application
- Advanced analytics dashboard

---

7. Appendix

This SRS document serves as the baseline for system design, development, and validation of the **Café Coffeeto – Smart Café Management System**.

III Project Plan - Gantt chart



Timeline mapping

Phase	Start Week	Duration
Requirement Analysis	Week 0	1 week
Design	Week 1	1 week
Implementation	Week 2	3 weeks
Testing	Week 5	1 week
Deployment	Week 6	1 week

IV Use Case Document

KEY USE CASE 1

Name: User Registration and Login

Identifier: UC-01

Description:

This use case allows a customer or admin to register for a new account or log in to an existing account to access the system features.

Goal:

To authenticate the user and allow access to the application.

Preconditions:

1. The system is online and accessible.

Assumptions:

1. The user has a valid email ID or phone number.

Frequency:

Very frequent.

Basic Course:

1. User opens the application.
2. System displays login and registration options.
3. User selects Register or Login.
4. User enters email/phone number and password.
5. System validates credentials.
6. User is logged in successfully.

Alternate Course A – Already Registered

Condition: Email ID or phone number already exists.

A.1 System displays an error message.

A.2 System prompts the user to log in.

Alternate Course B – Invalid Credentials

Condition: Incorrect login details.

B.1 System requests re-entry.

Postconditions:

1. User is authenticated.

Actors:

User, Admin

Included Use Cases:

None

Extended Use Cases:

Forgot Password (Optional)

KEY USE CASE 2

Name: Browse Menu

Identifier: UC-02

Description:

This use case allows users to browse café menu categories and view item details.

Goal:

To allow users to view and select menu items.

Preconditions:

1. User is authenticated or guest access is allowed.

Assumptions:

1. Menu data is available and updated.

Frequency:

Very frequent.

Basic Course:

1. System displays menu categories.
2. User browses items.
3. User views item details.

Postconditions:

1. Menu items are displayed.

Actors:

User

Included Use Cases:

None

Extended Use Cases:

Search Menu (Optional)

KEY USE CASE 3

Name: Place Order

Identifier: UC-03

Description:

Allows users to add items to cart and place an order.

Goal:

To place an order for selected items.

Preconditions:

1. User is authenticated.
2. Menu items are available.

Assumptions:

1. Inventory is sufficient.

Frequency:

Very frequent.

Basic Course:

1. User selects menu items.
2. System adds items to cart.
3. User reviews cart.
4. User confirms order.

Alternate Course – Item Out of Stock

Condition: Selected item is unavailable.

A.1 System notifies user.

A.2 User selects alternative.

Postconditions:

1. Order is created.

Actors:

User, System

Included Use Cases:

Browse Menu

Extended Use Cases:

Apply Coupon (Optional)

KEY USE CASE 4 (FINAL CONFIRMATION USE CASE)

Name: Make Payment

Identifier: UC-04

Description:

Allows users to pay for the placed order.

Goal:

To complete payment and confirm order.

Preconditions:

1. User is authenticated.
2. Order is placed.

Assumptions:

1. Payment gateway is available.

Frequency:

Very frequent.

Basic Course:

1. User proceeds to payment.
2. System shows payment options.
3. User selects method.
4. Payment is processed.
5. System confirms payment.

Alternate Course – Payment Failure

Condition: Payment fails.

A.1 System cancels transaction.

A.2 Order is not confirmed.

Postconditions:

1. Payment is successful.
2. Order is confirmed.

Actors:

User, Payment Gateway

Included Use Cases:

Place Order

Extended Use Cases:

Apply Discount (Optional)

KEY USE CASE 5(Business Use Case)

Name: Manage Inventory

Identifier: UC-05

Description:

Allows admin to manage stock levels of ingredients and items.

Goal:

To maintain accurate inventory data.

Preconditions:

1. Admin is authenticated.

Assumptions:

1. Inventory data exists.

Frequency:

Frequent.

Basic Course:

1. Admin views inventory dashboard.
2. Admin updates stock values.
3. System saves updates.

Postconditions:

1. Inventory is updated.

Actors:

Admin

Included Use Cases:

None

Extended Use Cases:

Low Stock Alert (Optional)

KEY USE CASE 6

Name: Track Order Status

Identifier: UC-06

Description:

Allows users to track order preparation and delivery status.

Goal:

To keep users informed about their orders.

Preconditions:

1. Order is placed.

Assumptions:

1. Staff updates order status.

Frequency:

Frequent.

Basic Course:

1. User opens order tracking page.
2. System displays current status.

Postconditions:

1. User sees updated order status.

Actors:

User, Staff

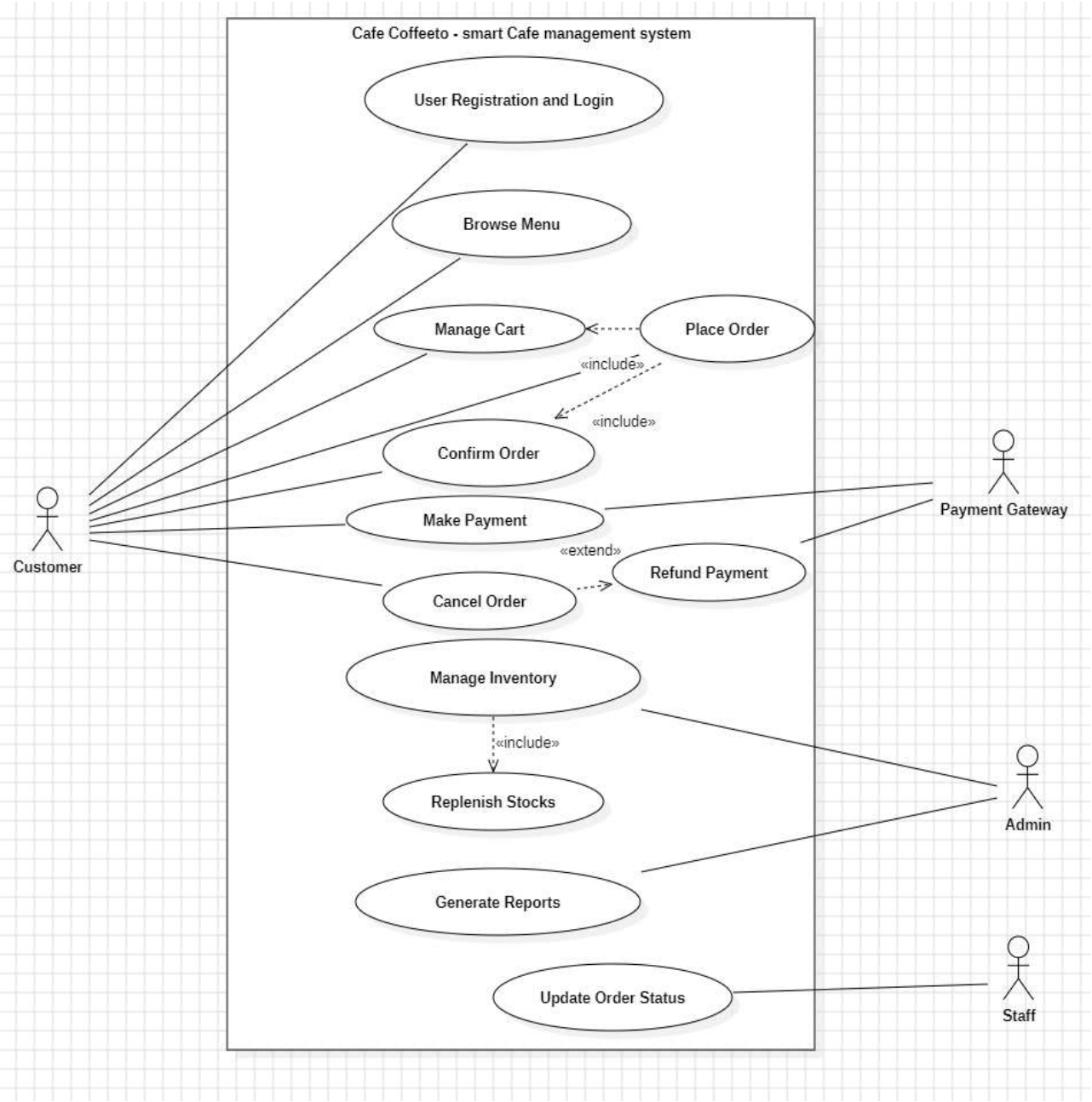
Included Use Cases:

Place Order

Extended Use Cases:

Receive Notification (Optional)

V Use Case Model



6. Identify the candidate classes with attributes and operations

User Management Module

1. User (Entity Class)

Attributes:

- userId : String
- name : String
- email : String
- phoneNumber : String
- password : String
- role : String

Operations:

- login()
- logout()
- updateProfile()

---

2. Customer (Entity Class)

Attributes:

- customerId : String
- address : String

Operations:

- browseMenu()
- placeOrder()
- trackOrder()

---

3. Admin (Entity Class)

Attributes:

- adminId : String

Operations:

- manageUsers()

- manageInventory()
- generateReports()

---

4. Staff (Entity Class)

Attributes:

- staffId : String
- shiftDetails : String

Operations:

- viewOrders()
- updateOrderStatus()

---

5. AuthService (Control Class)

Attributes:

- sessionId : String

Operations:

- authenticateUser()
- authorizeUser()
- resetPassword()

Order Management Module

1. Order (Entity Class)

Attributes

- orderId : String
- orderDate : Date
- status : String
- totalAmount : double

Operations

- createOrder()
- updateStatus()
- cancelOrder()
- calculateTotal()

---

2. OrderItem (Entity Class)

Attributes

- quantity : int
- price : double

Operations

- getSubtotal()

---

3. Cart (Entity Class)

Attributes

- cartId : String

Operations

- addItem()
- removeItem()
- clearCart()

---

4. MenuItem (Entity Class)

Attributes

- itemId : String
- name : String
- price : double
- availability : boolean

Operations

- updateAvailability()

---

5. OrderController (Control Class)

Attributes

- controllerId : String

Operations

- placeOrder()
- validateOrder()
- notifyUser()

Payment Module

1. Payment (Entity Class)

Attributes

- paymentId : String
- amount : double
- paymentDate : Date
- paymentStatus : String
- paymentMode : String

Operations

- initiatePayment()
- updatePaymentStatus()
- generateReceipt()

---

2. Invoice (Entity Class)

Attributes

- invoiceId : String
- invoiceDate : Date
- totalAmount : double

Operations

- generateInvoice()
- printInvoice()

---

3. PaymentGateway (Interface / External System)

Attributes

- gatewayName : String

Operations

- processPayment()
- refundPayment()

---

4. PaymentService (Control Class)

Attributes

- serviceId : String

Operations

- validatePayment()
- processTransaction()
- confirmPayment()

Inventory Module

1. Inventory (Entity Class)

Attributes

- inventoryId : String
- lastUpdated : Date

Operations

- checkStock()
- updateStock()
- getLowStockItems()

---

2. Item (Entity Class)

Attributes

- itemId : String
- itemName : String
- quantity : int
- thresholdLevel : int

Operations

- increaseQuantity()
- decreaseQuantity()

---

3. Supplier (Entity Class)

Attributes

- supplierId : String
- supplierName : String
- contactNumber : String

Operations

- supplyItems()

---

4. InventoryController (Control Class)

Attributes

- controllerId : String

Operations

- monitorStock()
- generateLowStockAlert()
- requestReplenishment()

Delivery Module

1. Delivery (Entity Class)

Attributes

- deliveryId : String
- deliveryStatus : String
- deliveryAddress : String
- deliveryTime : DateTime

Operations

- assignDeliveryPartner()
- updateDeliveryStatus()
- getDeliveryDetails()

---

2. Order (Entity Class)

Attributes

- orderId : String
- orderStatus : String

Operations

- getOrderDetails()

---

3. DeliveryPartner (Boundary Class)

Attributes

- partnerId : String
- partnerName : String
- contactNumber : String

Operations

- acceptDelivery()
- updateDeliveryProgress()

---

4. DeliveryService (Control Class)

Attributes

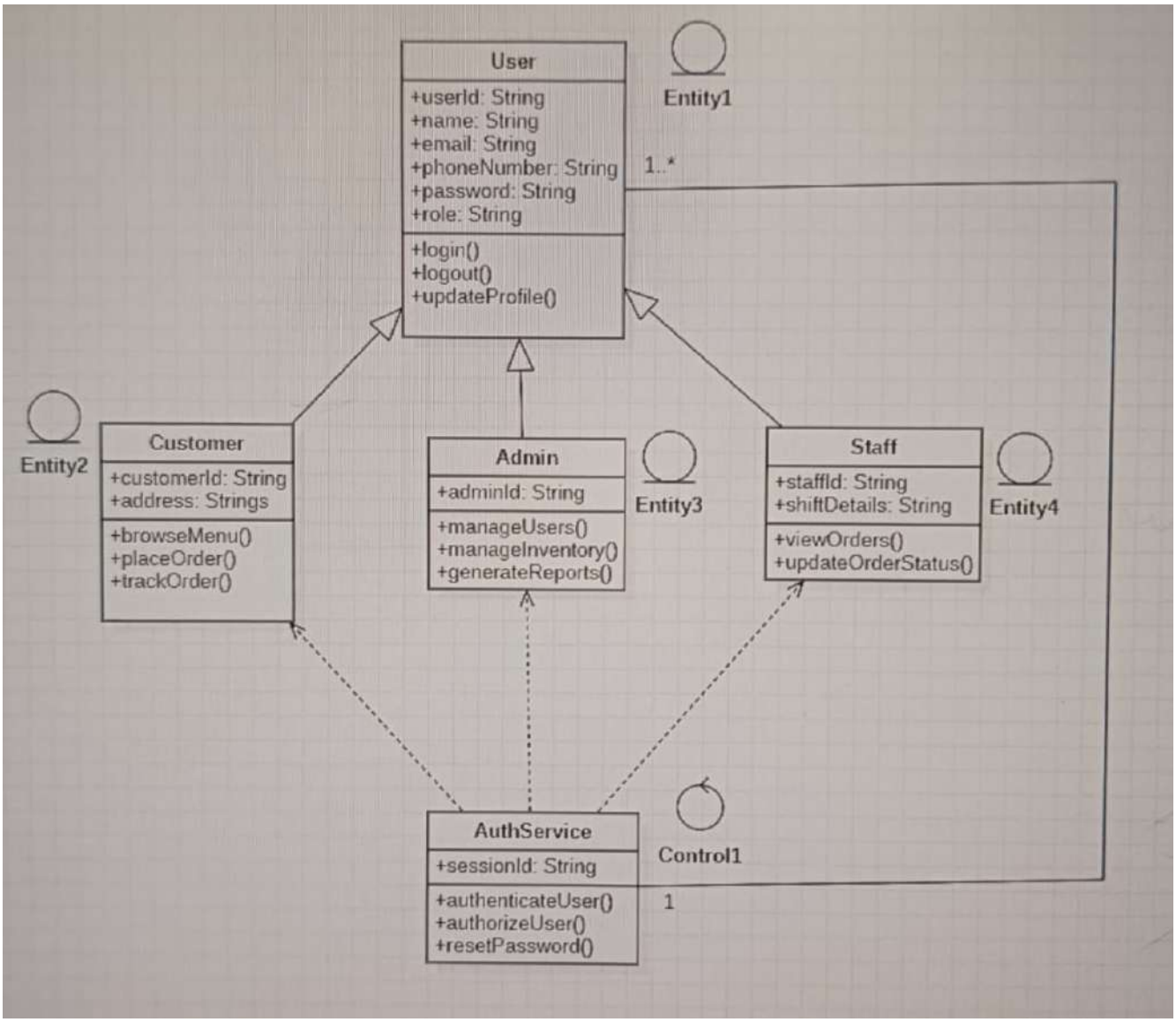
- serviceId : String

Operations

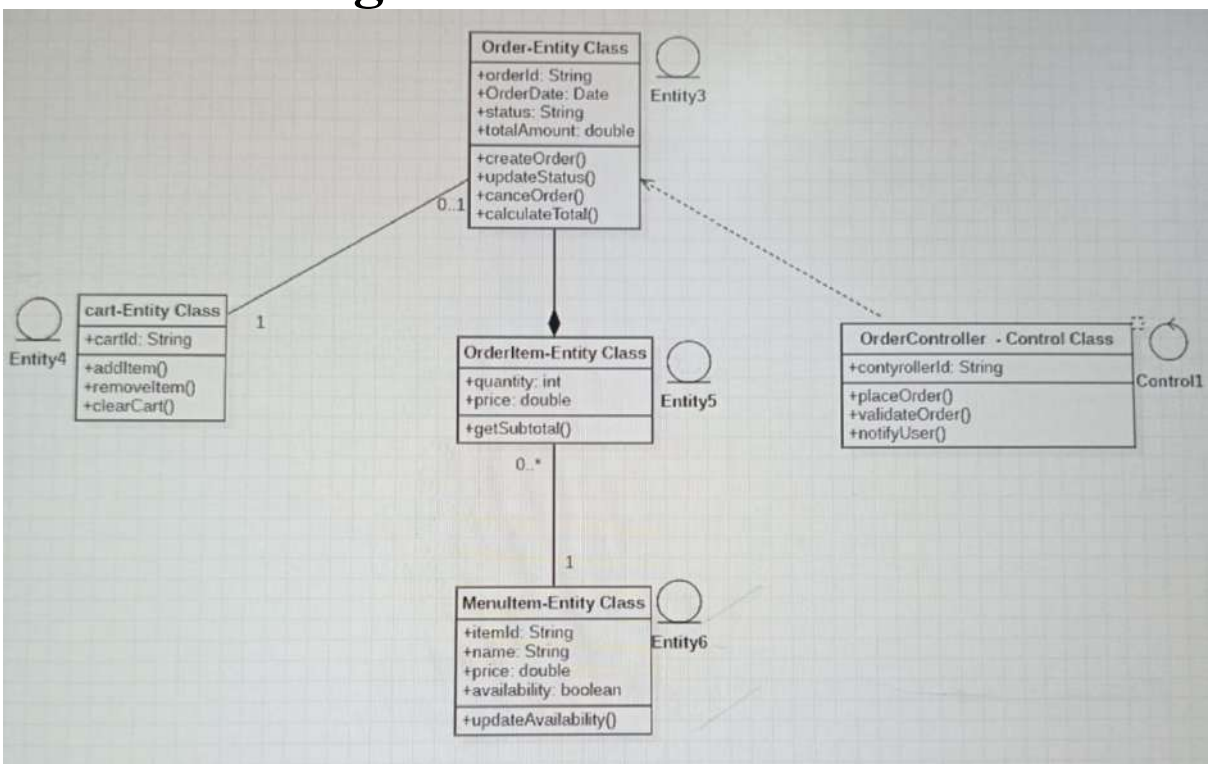
- scheduleDelivery()
- notifyCustomer()
- trackDelivery()

7. Develop the class diagram for each module and apply the following relations.

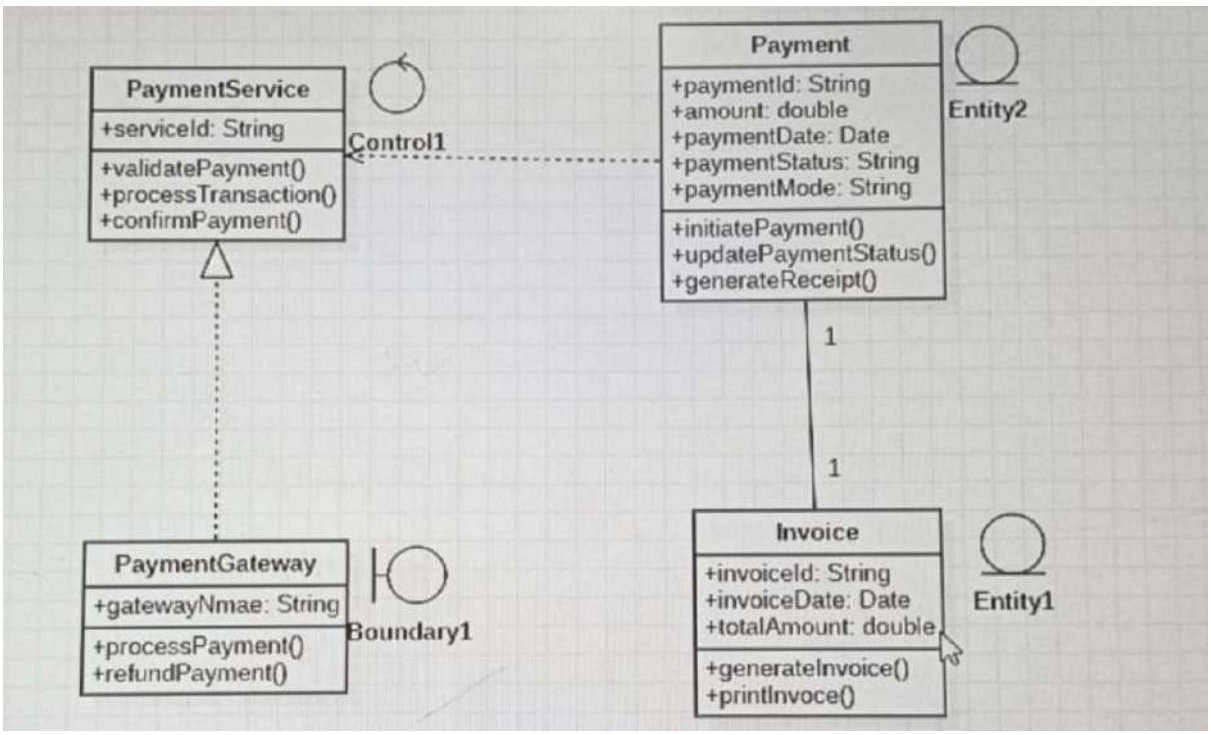
User Management Module



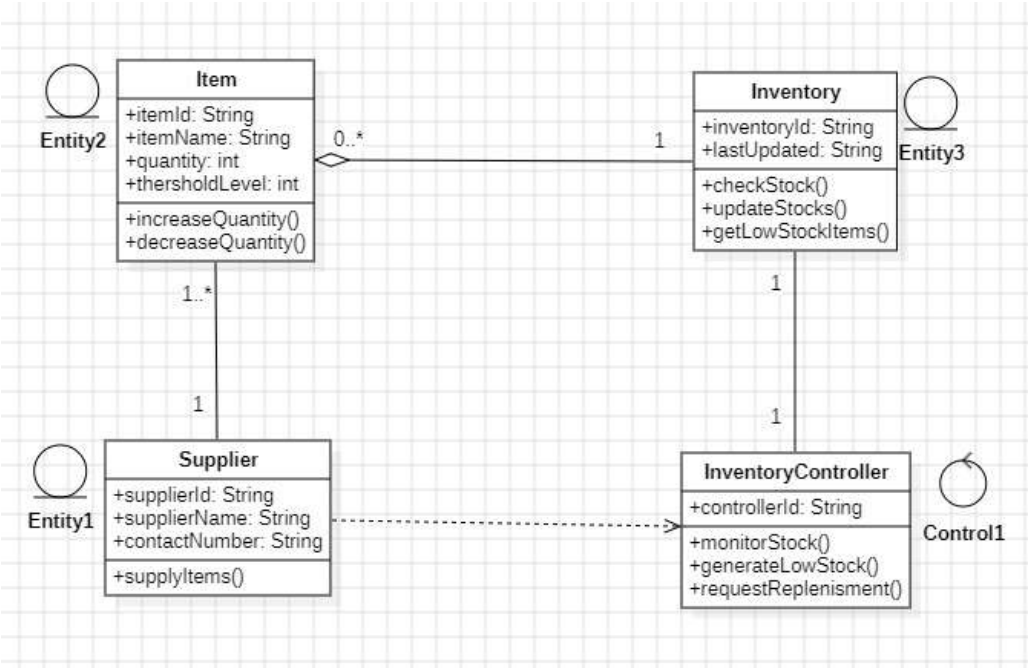
Order Management Module



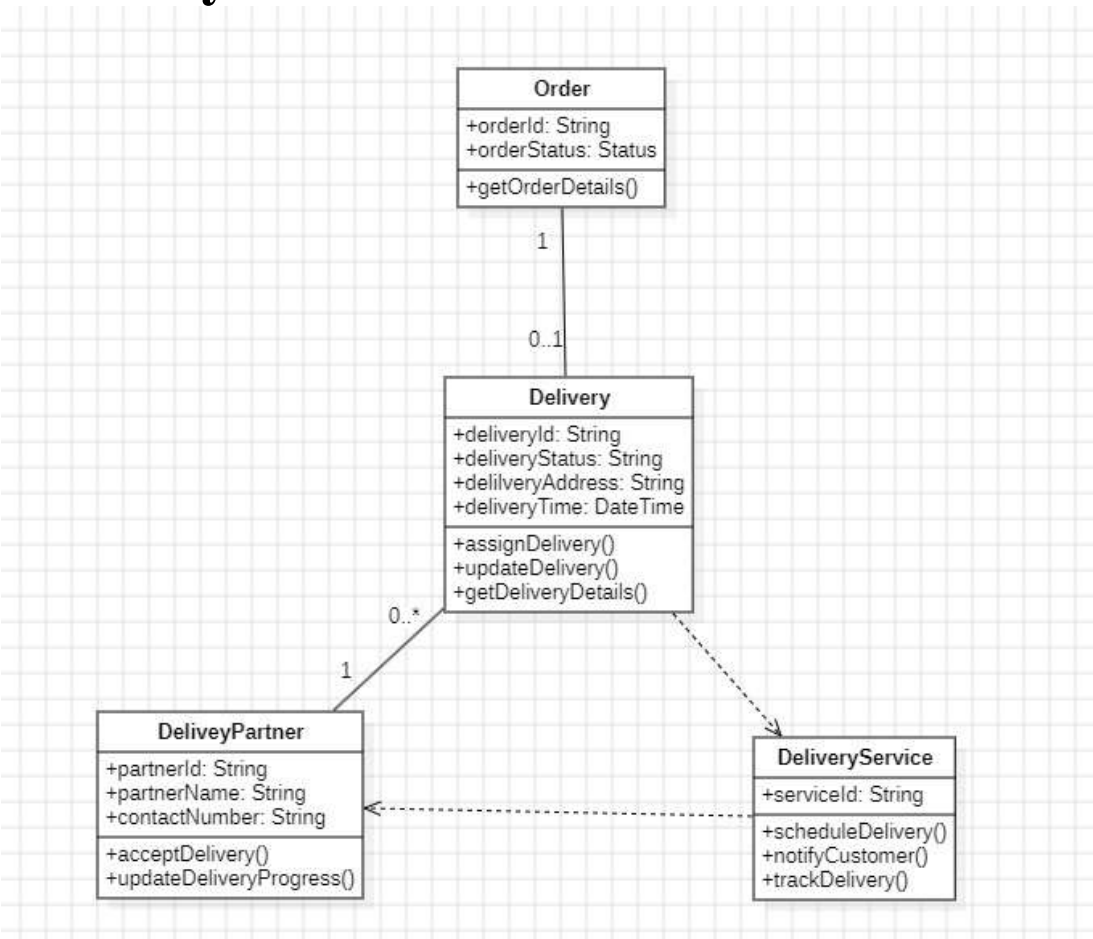
Payment Module



Inventory Module

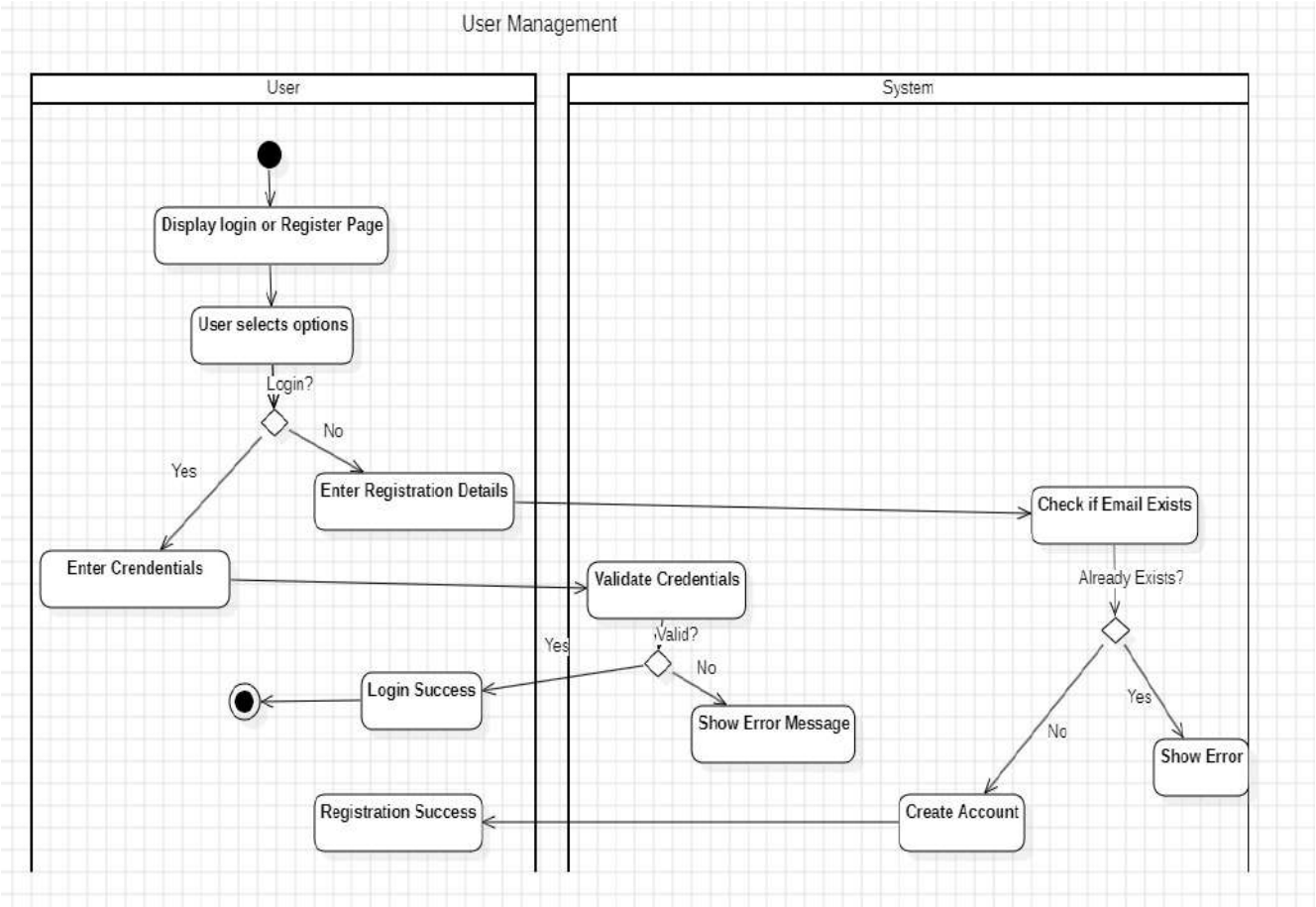


Delivery Module

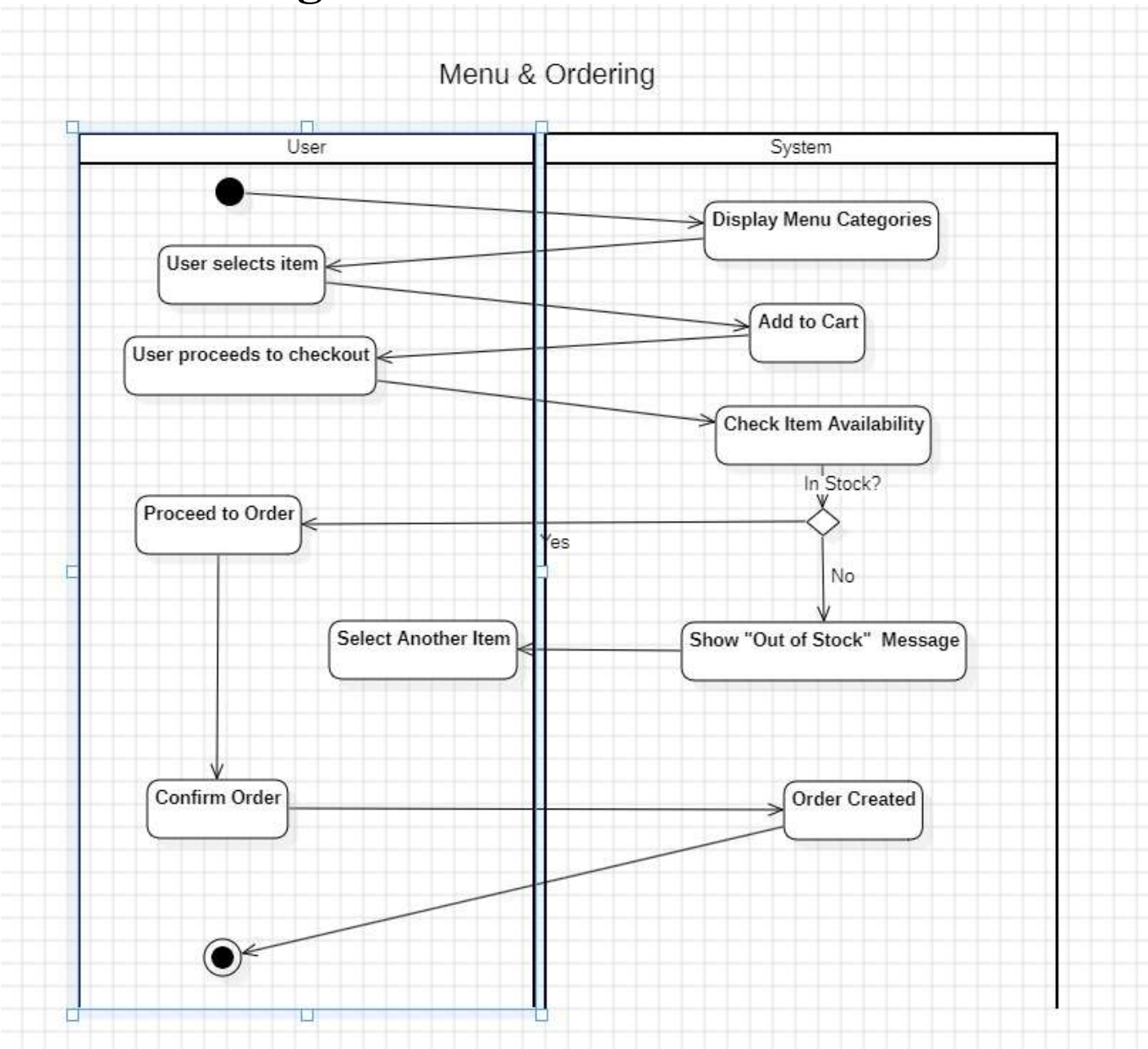


3. Develop the activity diagram for each module identified with all notations.Show all the alternate flows identified in use case document.

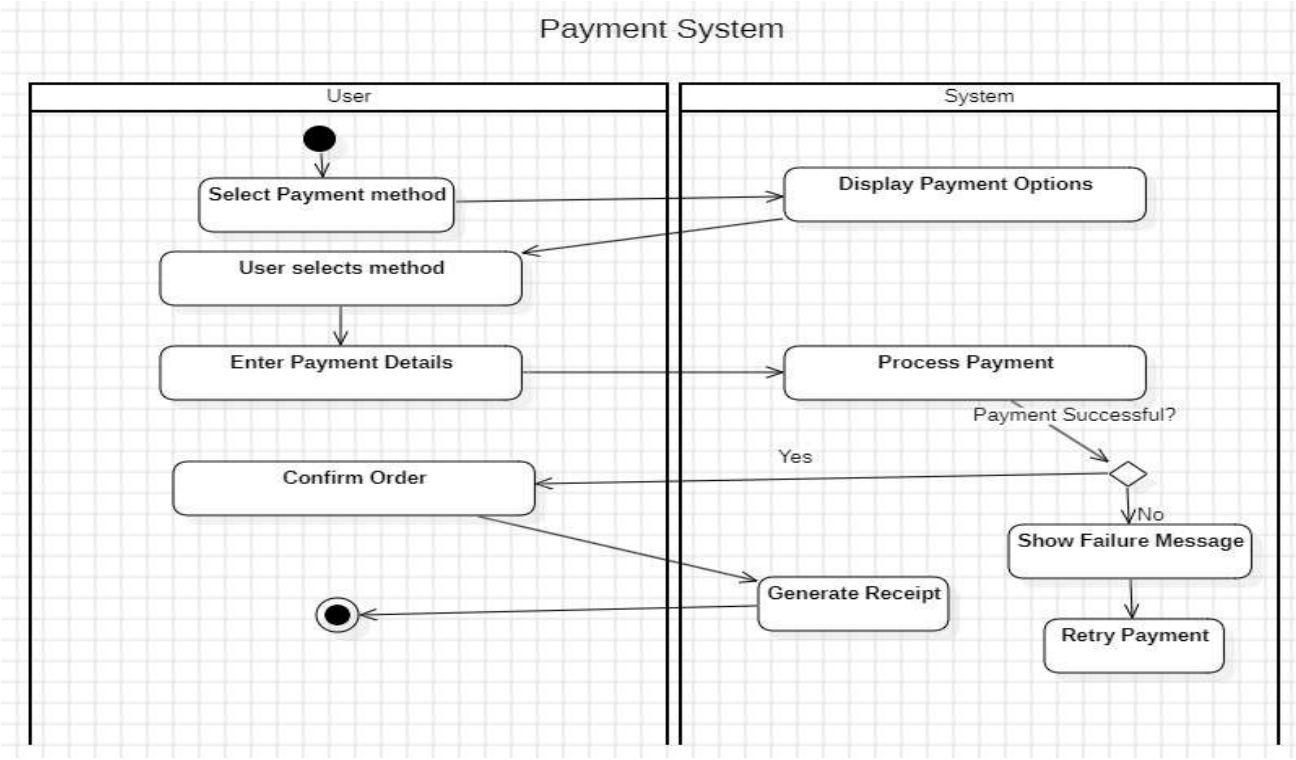
User Management Module


User Management
User
System
Display login or Register Page
User selects options
Login?
Yes
No
Enter Registration Details
Enter Crendentials
Check if Email Exists
Validate Credentials
Already Exists?
Valid?
Yes
No
Login Success
Show Error Message
Yes
No
Show Error
Registration Success
Create Account

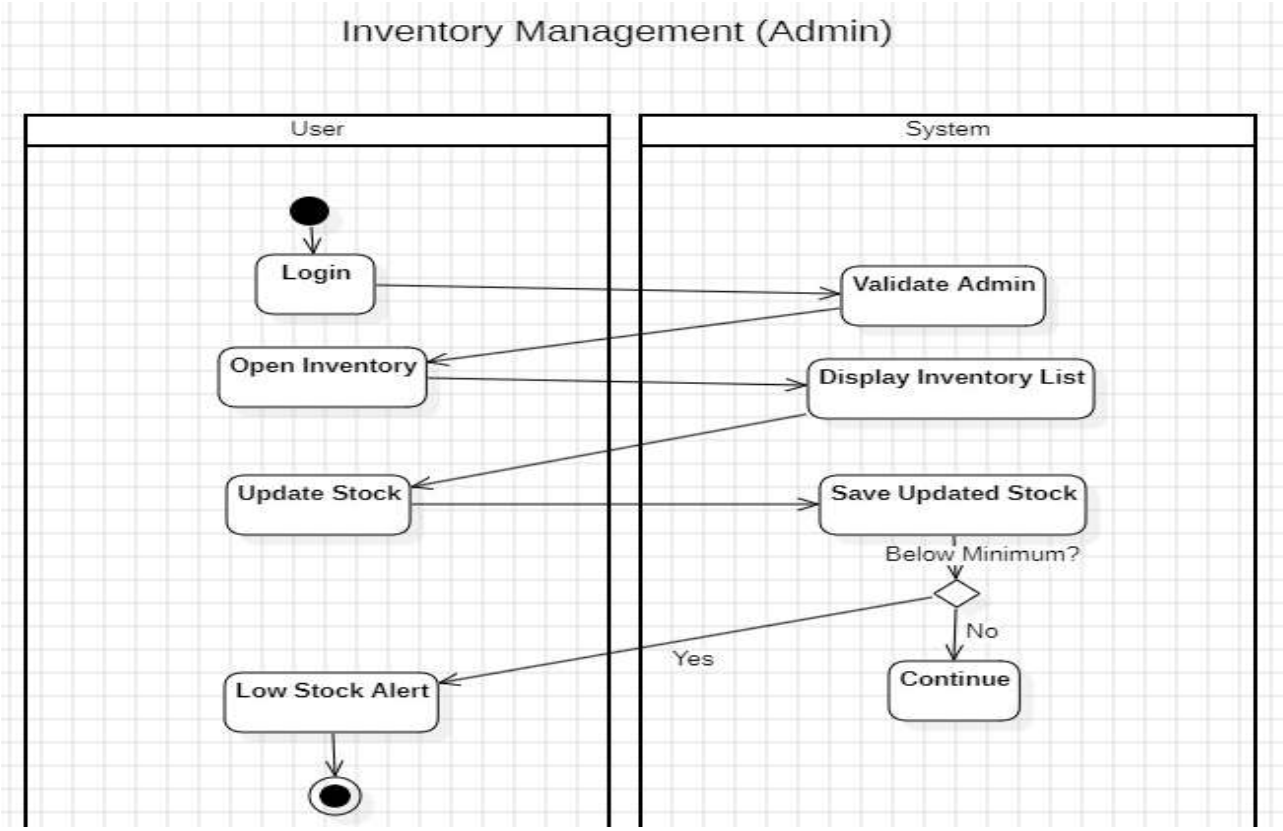
Order Management Module



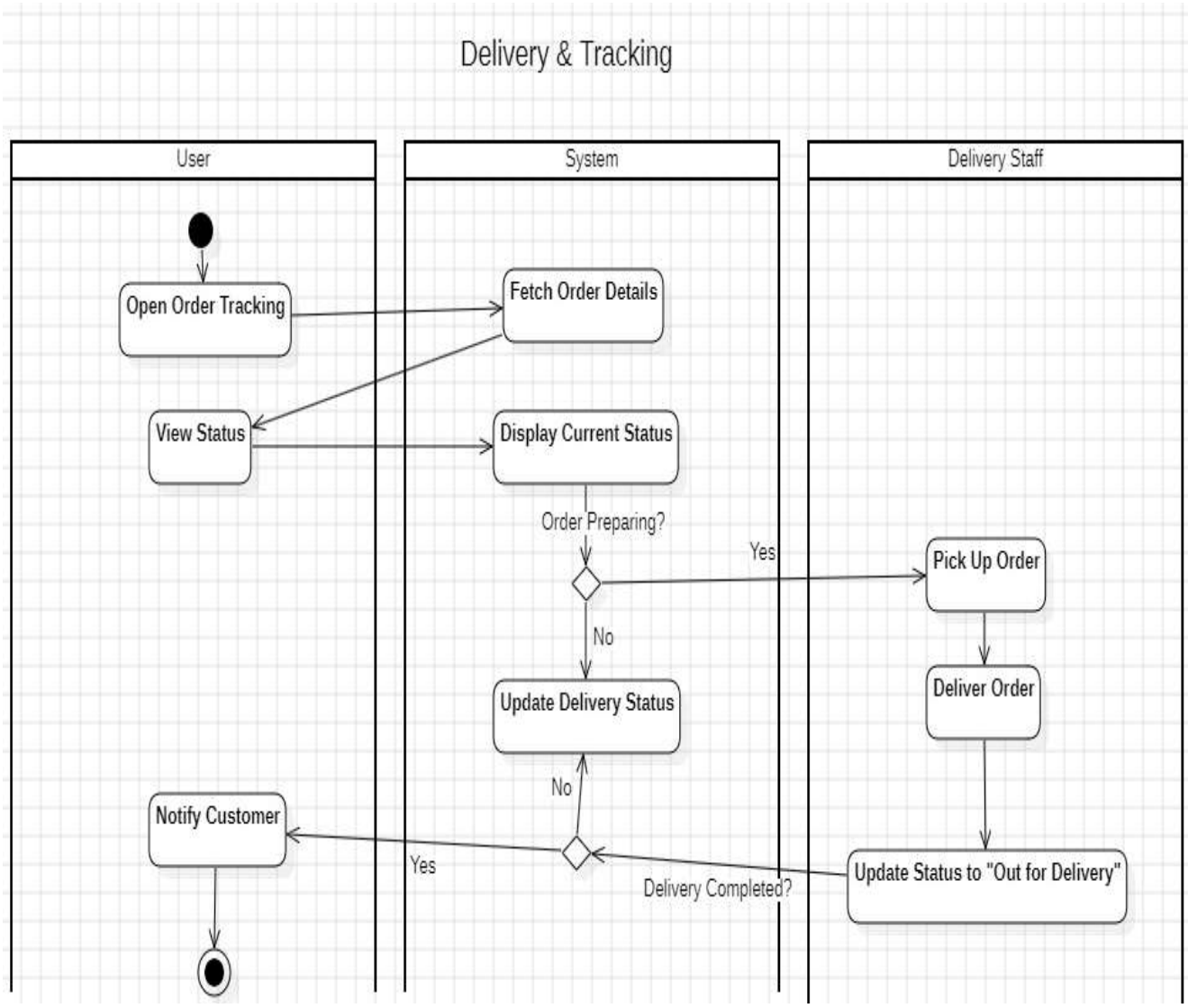
Payment Module



Inventory Module

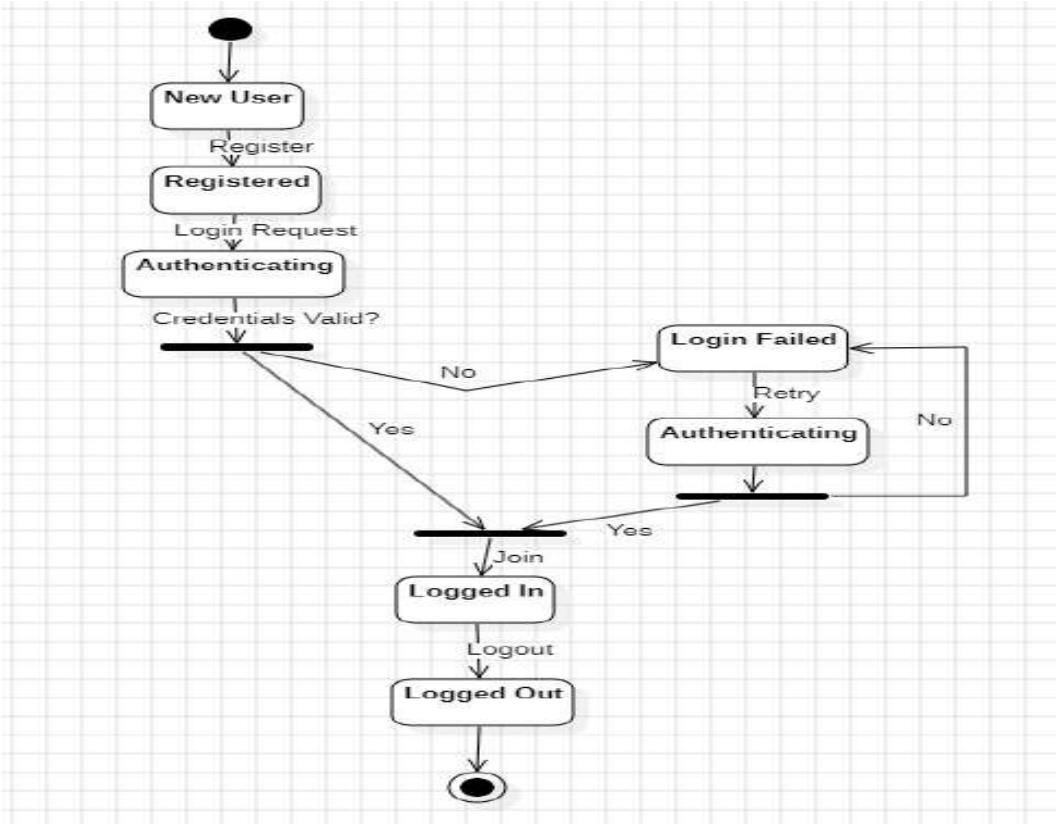


Delivery Module


Delivery & Tracking
User
System
Delivery Staff
Open Order Tracking
Fetch Order Details
View Status
Display Current Status
Order Preparing?
Yes
Pick Up Order
No
Deliver Order
Update Delivery Status
No
Notify Customer
Yes
Delivery Completed?
Update Status to "Out for Delivery"

4. Construct the state chart diagram for each modules Show the state ,stimuli, response for the identified entity in each module

User Management



Simple States

- New User
- Registered
- Logged In
- Logged Out
- Login Failed

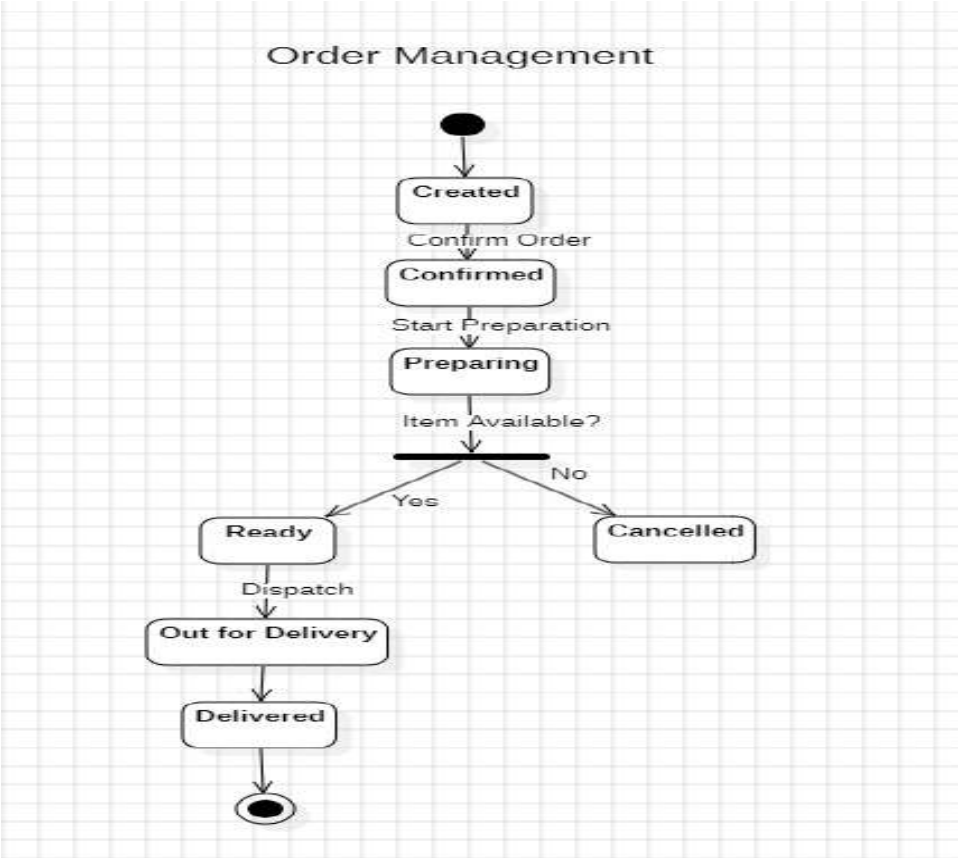
Composite States

- Authenticating

Submachine States

- Authentication Process

Order Management Module



Simple States

- Created
- Confirmed
- Ready
- Delivered
- Cancelled

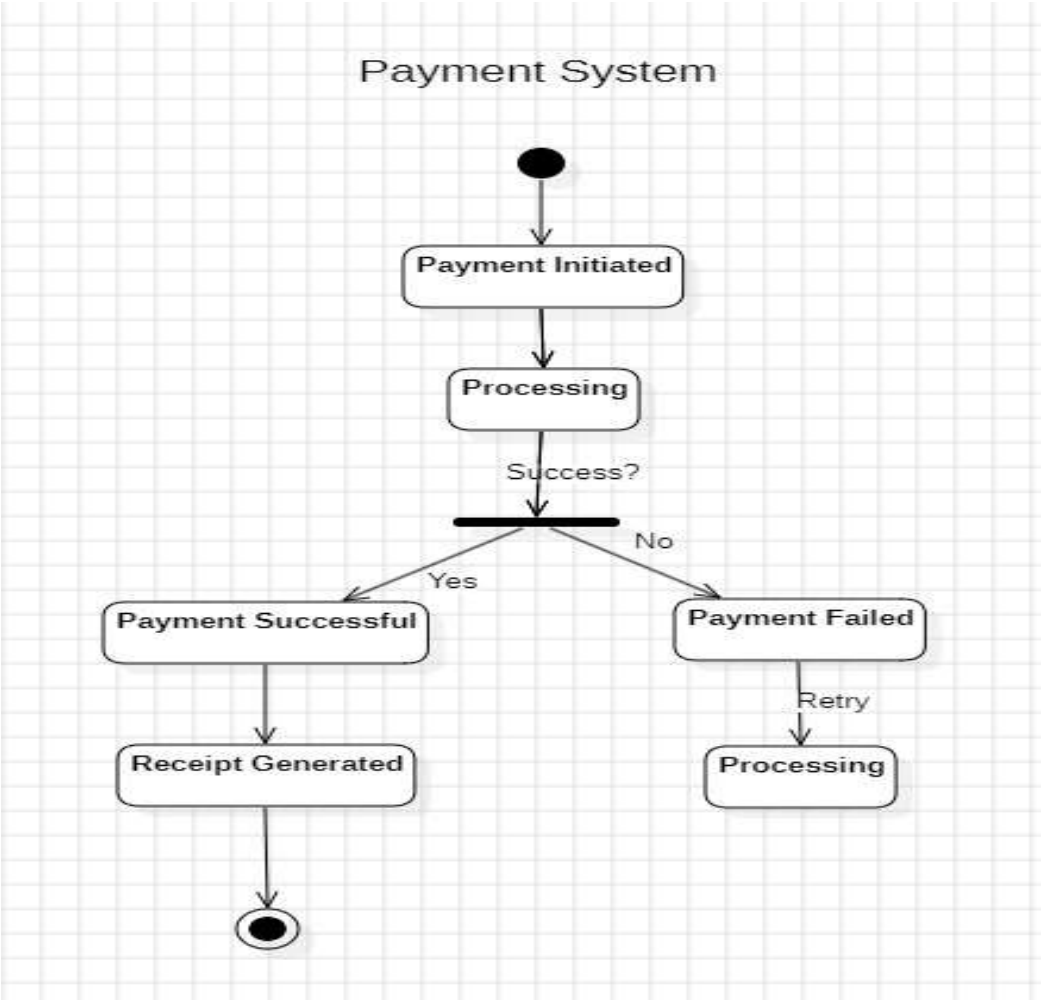
Composite States

- Preparing

Submachine States

- Delivery Process

Payment Module



Simple States

- Payment Initiated
- Payment Successful
- Payment Failed
- Receipt Generated

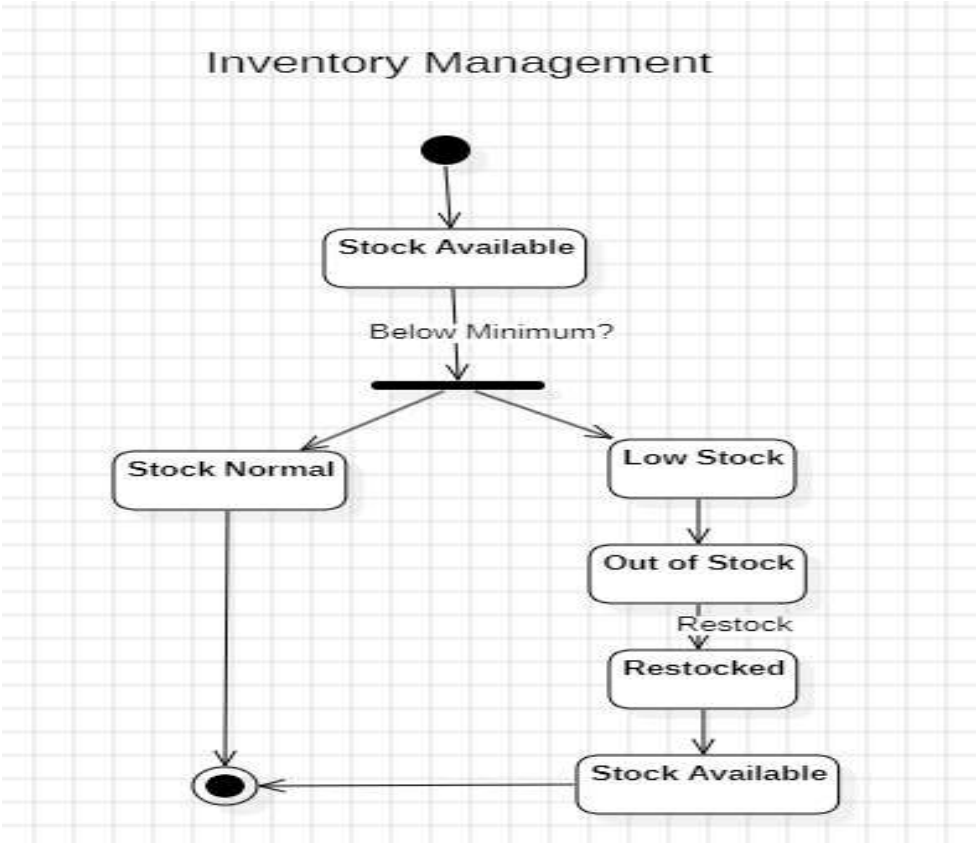
Composite State

- Processing

Submachine Stat

- Payment Gateway Interaction

Inventory Module



Simple States

- Stock Normal
- Low Stock
- Out of Stock
- Restocked

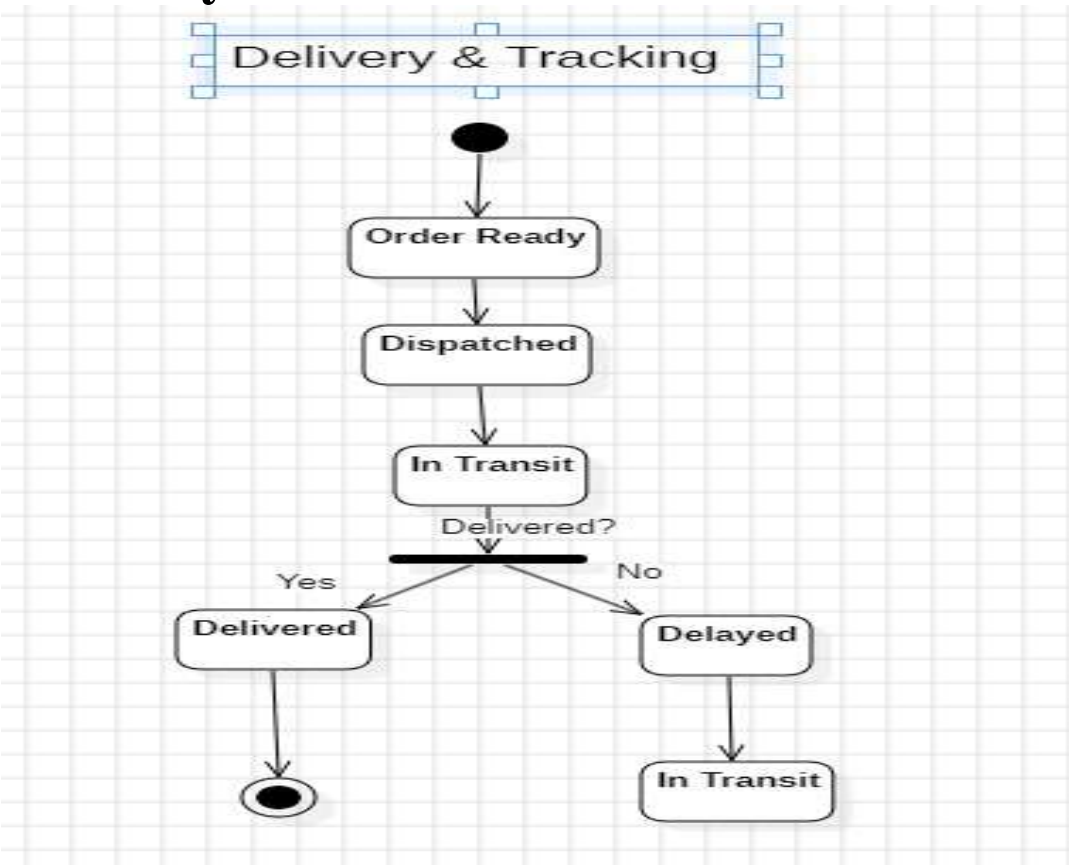
Composite States

- Stock Available

Submachine States

- Replenishment Process

Delivery Module



Simple States

- Order Ready
- Dispatched
- Delivered

Composite States

- In Transit

Submachine States

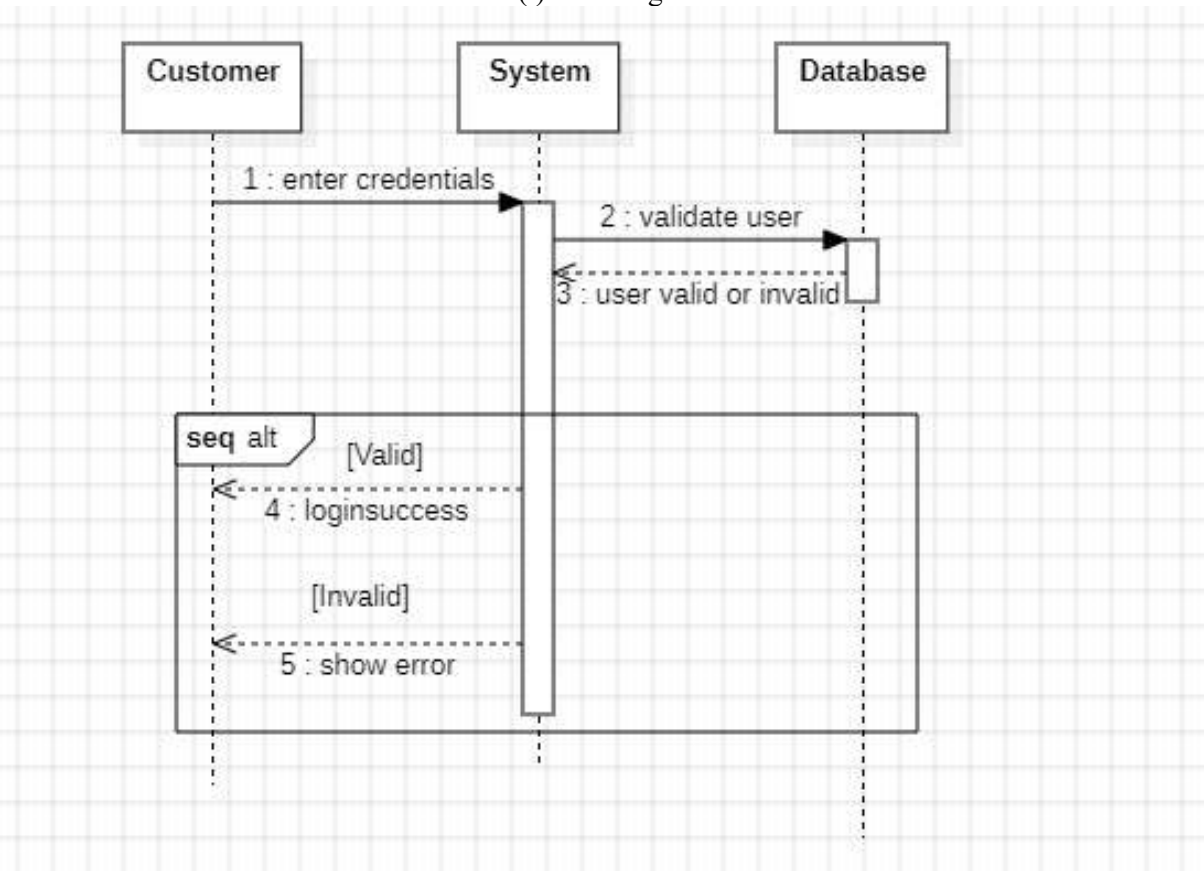
- Tracking System

8. Develop the sequence diagram using the following

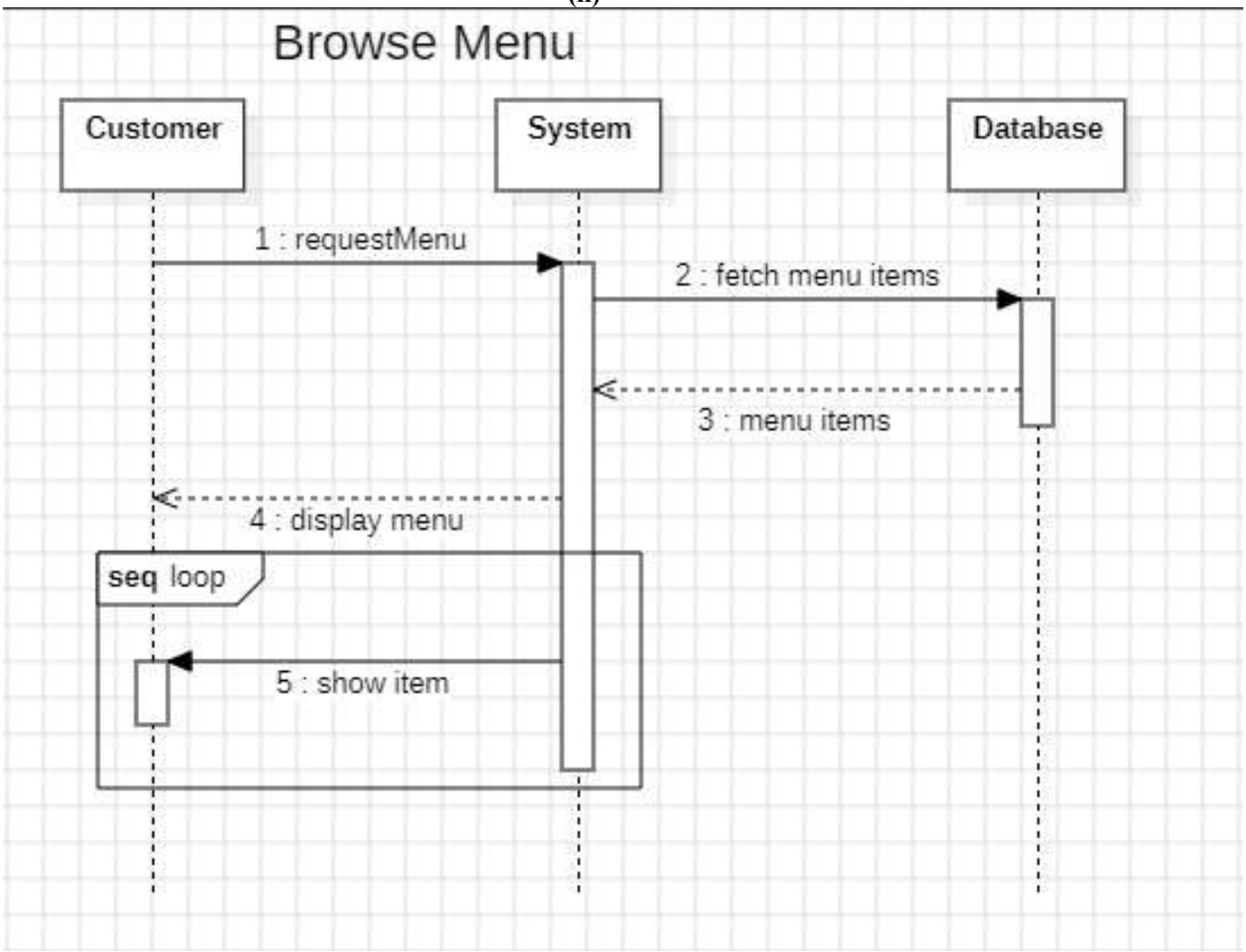
• Script for every key use case

• Combined fragment (all fragment types)

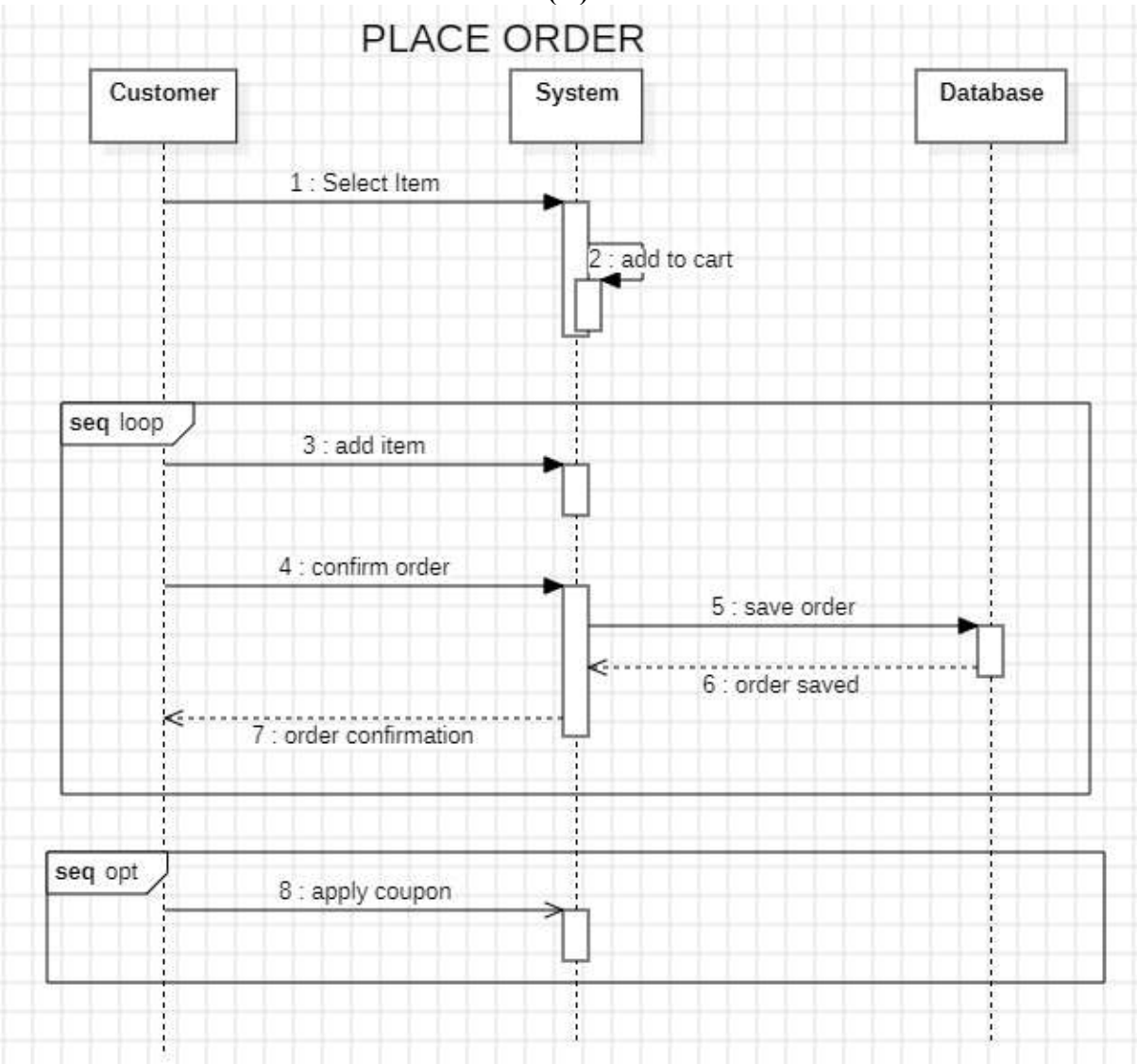
(i) User Login



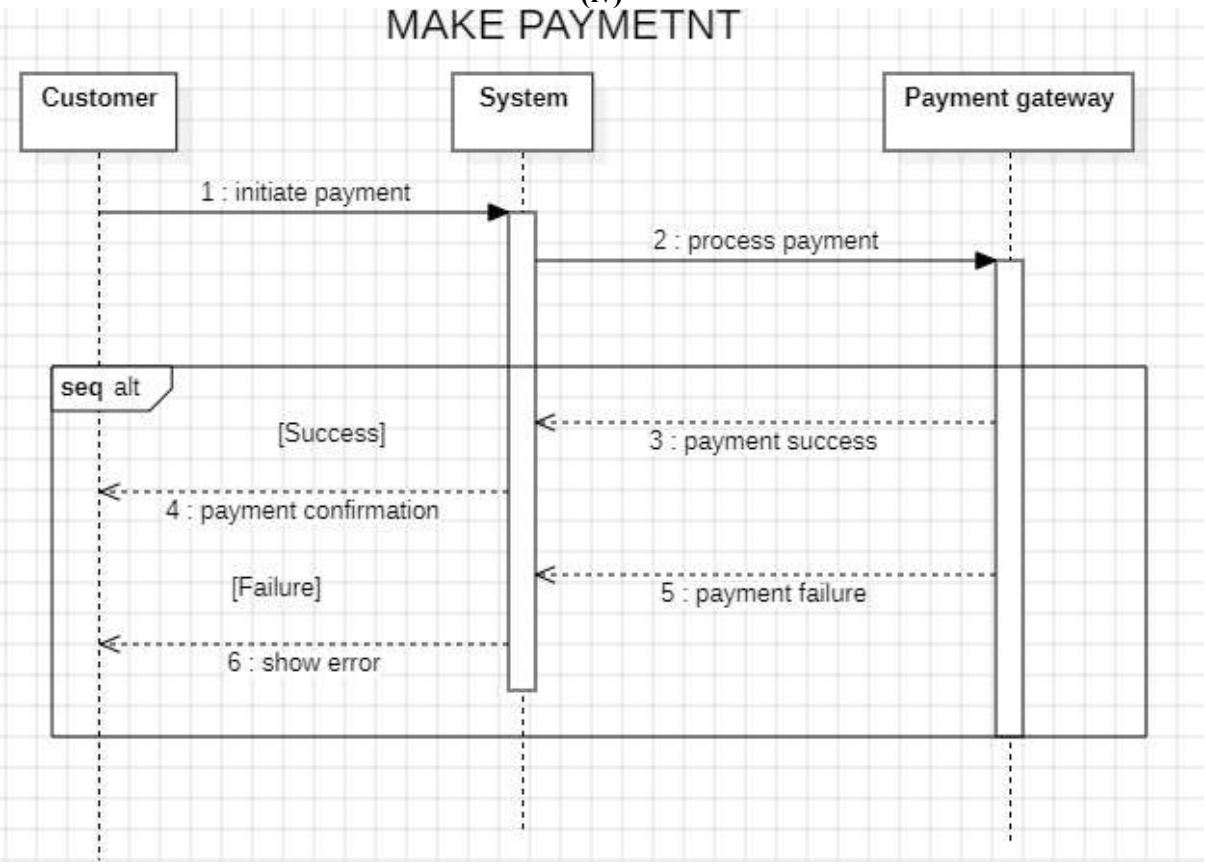
(ii)


Browse Menu
Customer
System
Database
1 : requestMenu
2 : fetch menu items
3 : menu items
4 : display menu
seq loop
5 : show item

(iii)

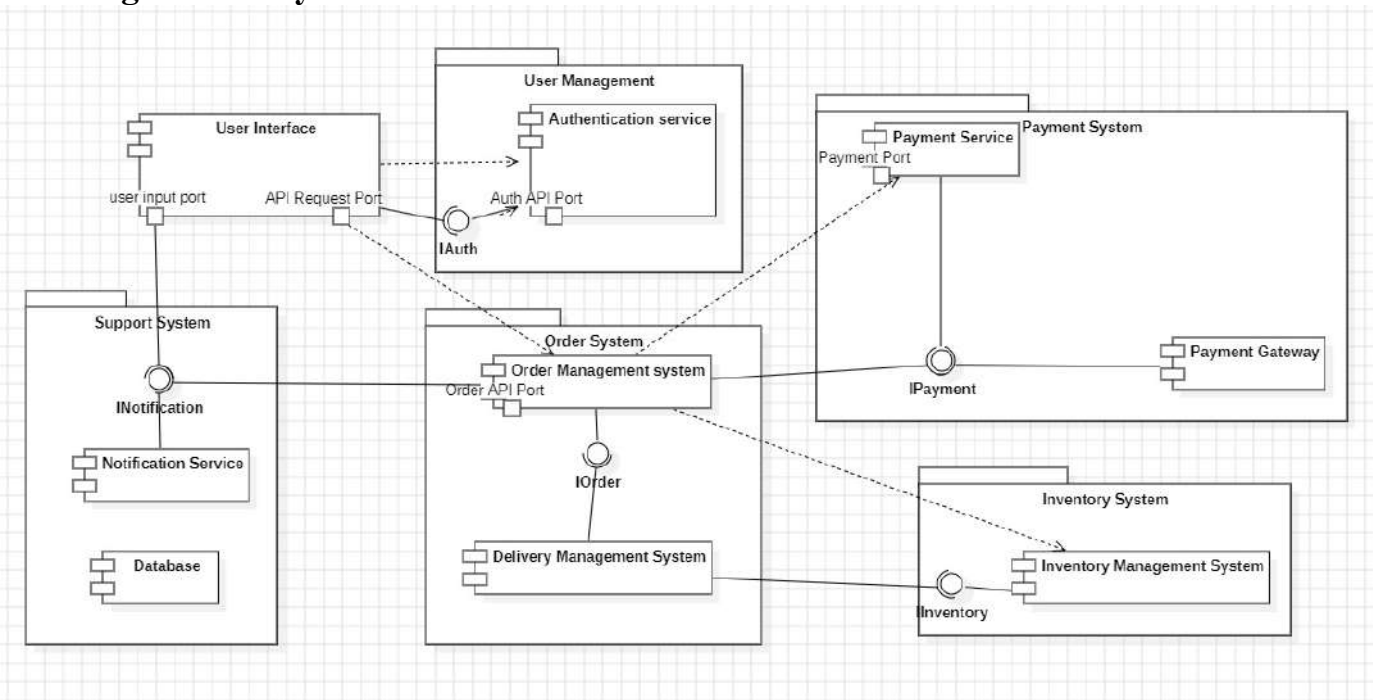

PLACE ORDER
Customer
System
Database
1 : Select Item
2 : add to cart
seq loop
3 : add item
4 : confirm order
5 : save order
6 : order saved
7 : order confirmation
seq opt
8 : apply coupon

(iv)


MAKE PAYMETNT
Customer
System
Payment gateway
1 : initiate payment
2 : process payment
seq alt
[Success]
3 : payment success
4 : payment confirmation
[Failure]
5 : payment failure
6 : show error

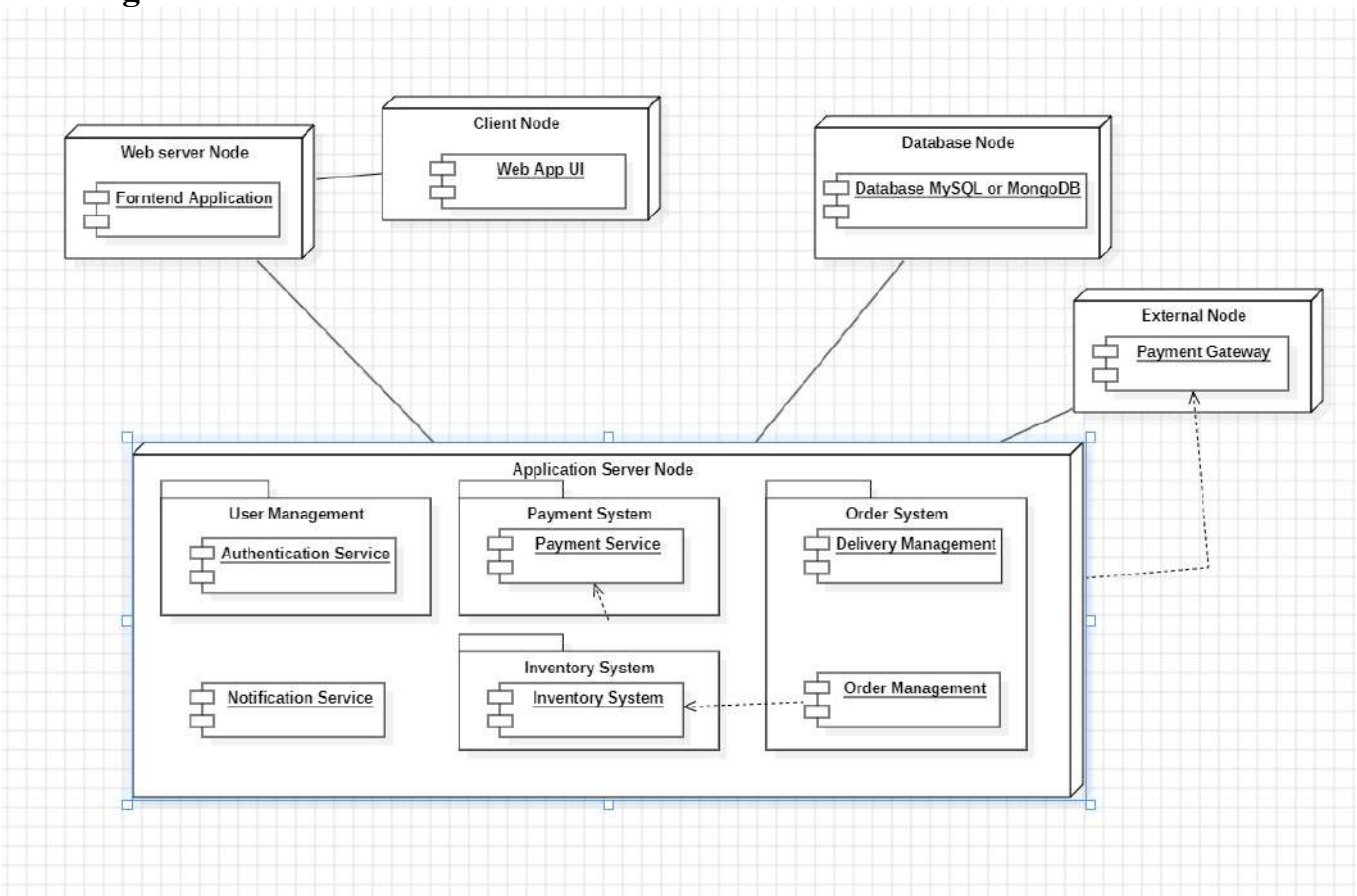
9. Develop the Component Diagram with the following

- **Components**
- **Interfaces**
- **Dependency, generalization, association, and realization**
- **Relationships**
- **Ports and interaction point between a component and its environment**
- **Notes and constraints**
- **Packages or subsystems**

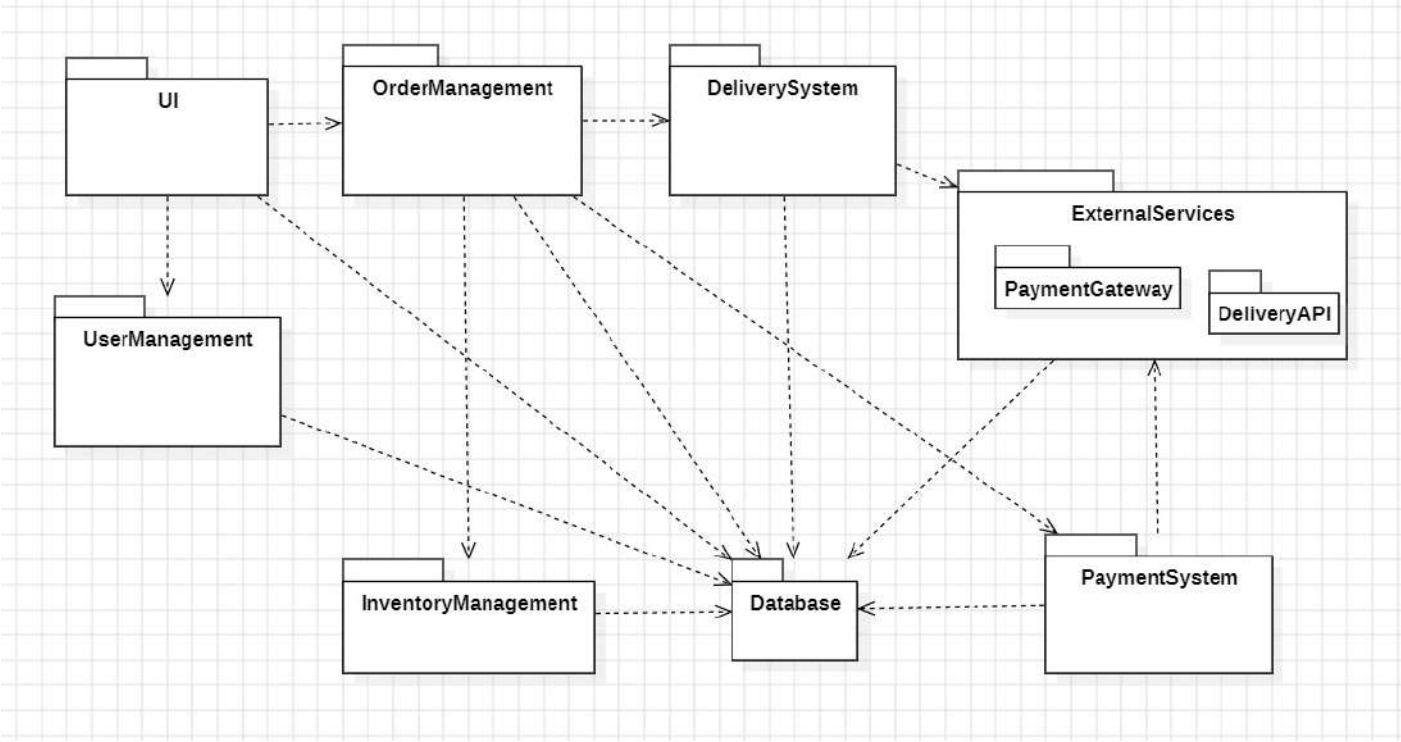


10. Show the deployment diagram with the following

- **Nodes**
- **Dependency and association relationships**
- **Notes and constraints**
- **Components**
- **Packages**



11. Outline the package diagram.



12. Generate the code for the classes identified.

(i) User Management Module

```
// Base Class
class User {
   protected String userId;
   protected String name;
   protected String email;
   protected String phoneNumber;
   protected String password;
   protected String role;

   public void login() {
      System.out.println("User logged in");
   }

   public void logout() {
      System.out.println("User logged out");
   }

   public void updateProfile() {
      System.out.println("Profile updated");
   }
}

// Customer
class Customer extends User {
   private String customerId;
   private String address;

   public void browseMenu() {}
   public void placeOrder() {}
   public void trackOrder() {}
}

// Admin
class Admin extends User {
   private String adminId;
```

```
    public void manageUsers() {}
    public void manageInventory() {}
    public void generateReports() {}
}

// Staff
class Staff extends User {
    private String staffId;
    private String shiftDetails;

    public void viewOrders() {}
    public void updateOrderStatus() {}
}

// Control Class
class AuthService {
    private String sessionId;

    public boolean authenticateUser(String email, String password) {
        return true;
    }

    public void authorizeUser() {}
    public void resetPassword() {}
}
```

(ii) Order Management Module

```
import java.util.*;

class Order {
    private String orderId;
    private Date orderDate;
    private String status;
    private double totalAmount;
    private List<OrderItem> items = new ArrayList<>();

    public void createOrder() {}
    public void updateStatus() {}
    public void cancelOrder() {}
```

```
    public double calculateTotal() {
        return items.stream().mapToDouble(OrderItem::getSubtotal).sum();
    }
}

class OrderItem {
    private int quantity;
    private double price;

    public double getSubtotal() {
        return quantity * price;
    }
}

class Cart {
    private String cartId;
    private List<MenuItem> items = new ArrayList<>();

    public void addItem(MenuItem item) {
        items.add(item);
    }

    public void removeItem(MenuItem item) {
        items.remove(item);
    }

    public void clearCart() {
        items.clear();
    }
}

class MenuItem {
    private String itemId;
    private String name;
    private double price;
    private boolean availability;

    public void updateAvailability(boolean status) {
        this.availability = status;
```

```
    }
}

// Control Class
class OrderController {
    private String controllerId;

    public void placeOrder() {}
    public void validateOrder() {}
    public void notifyUser() {}
}
```

(iii) Payment Module

```
import java.util.Date;

// Interface (Boundary)
interface PaymentGateway {
    void processPayment(double amount);
    void refundPayment(double amount);
}

// Entity
class Payment {
    private String paymentId;
    private double amount;
    private Date paymentDate;
    private String paymentStatus;
    private String paymentMode;

    public void initiatePayment() {}
    public void updatePaymentStatus() {}
    public void generateReceipt() {}
}

// Entity
class Invoice {
    private String invoiceId;
    private Date invoiceDate;
    private double totalAmount;
```

```
    public void generateInvoice() {}
    public void printInvoice() {}
}

// Control Class
class PaymentService implements PaymentGateway {
    private String serviceId;

    public void validatePayment() {}

    public void processTransaction() {}

    public void confirmPayment() {}

    @Override
    public void processPayment(double amount) {
        System.out.println("Processing payment...");
    }

    @Override
    public void refundPayment(double amount) {
        System.out.println("Refund processed...");
    }
}
```

(iv) Inventory Module

```
import java.util.*;

class Inventory {
    private String inventoryId;
    private Date lastUpdated;
    private List<Item> items = new ArrayList<>();

    public void checkStock() {}

    public void updateStock() {}

    public List<Item> getLowStockItems() {
```

```
        List<Item> lowStock = new ArrayList<>();
        for (Item item : items) {
            if (item.getQuantity() < item.getThresholdLevel()) {
                lowStock.add(item);
            }
        }
        return lowStock;
    }
}

class Item {
    private String itemId;
    private String itemName;
    private int quantity;
    private int thresholdLevel;

    public void increaseQuantity(int amount) {
        quantity += amount;
    }

    public void decreaseQuantity(int amount) {
        quantity -= amount;
    }

    public int getQuantity() {
        return quantity;
    }

    public int getThresholdLevel() {
        return thresholdLevel;
    }
}

class Supplier {
    private String supplierId;
    private String supplierName;
    private String contactNumber;

    public void supplyItems() {}
}
```

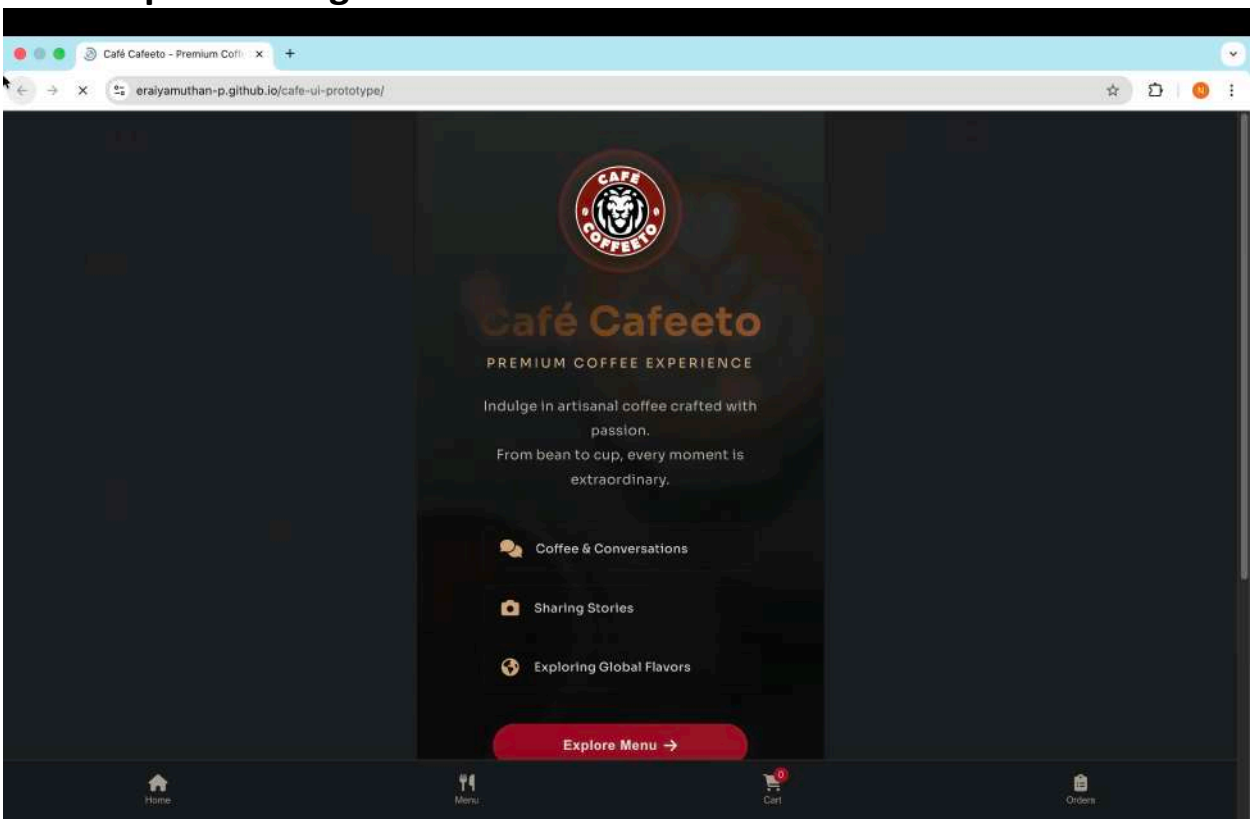
```
// Control Class
class InventoryController {
   private String controllerId;

   public void monitorStock() {}
   public void generateLowStockAlert() {}
   public void requestReplenishment() {}
}
```

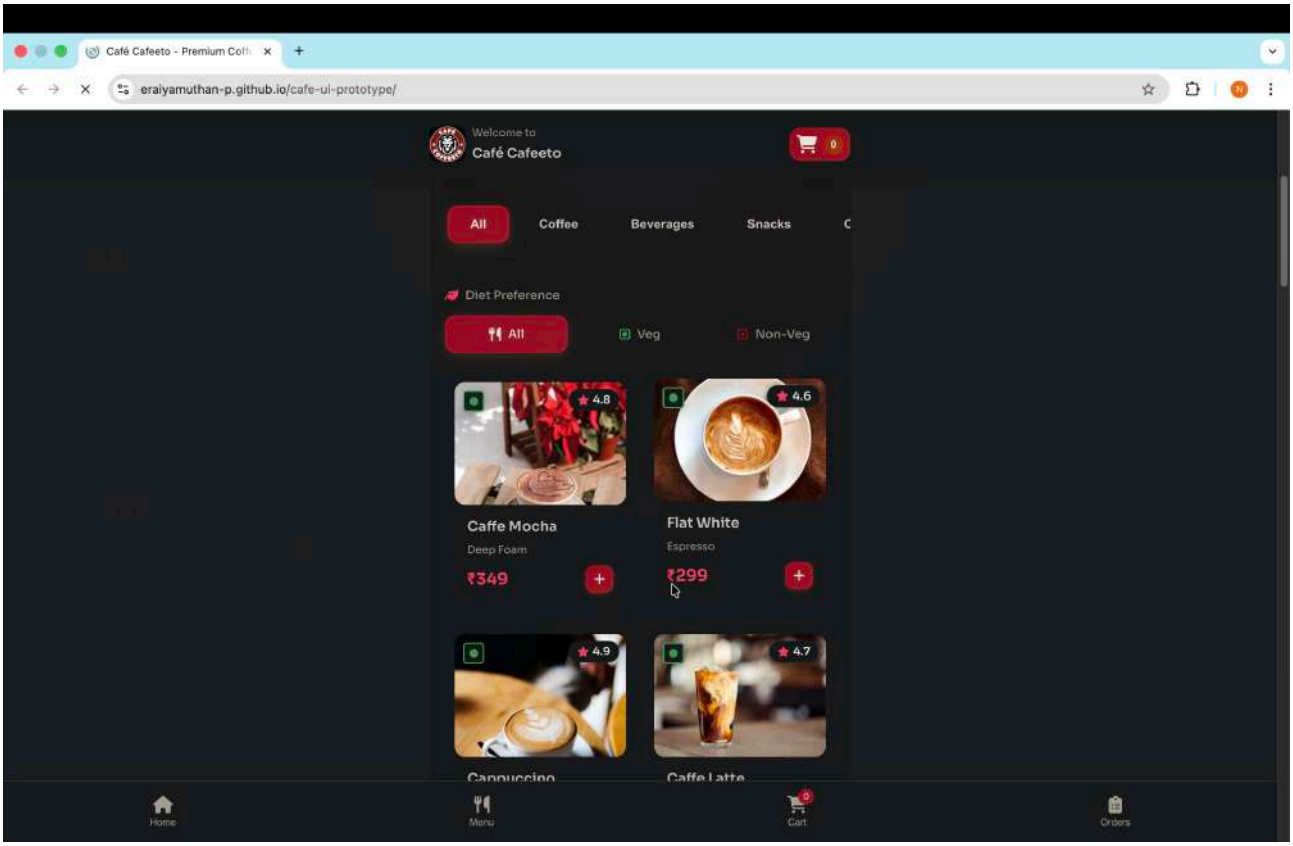
(v) Delivery Module

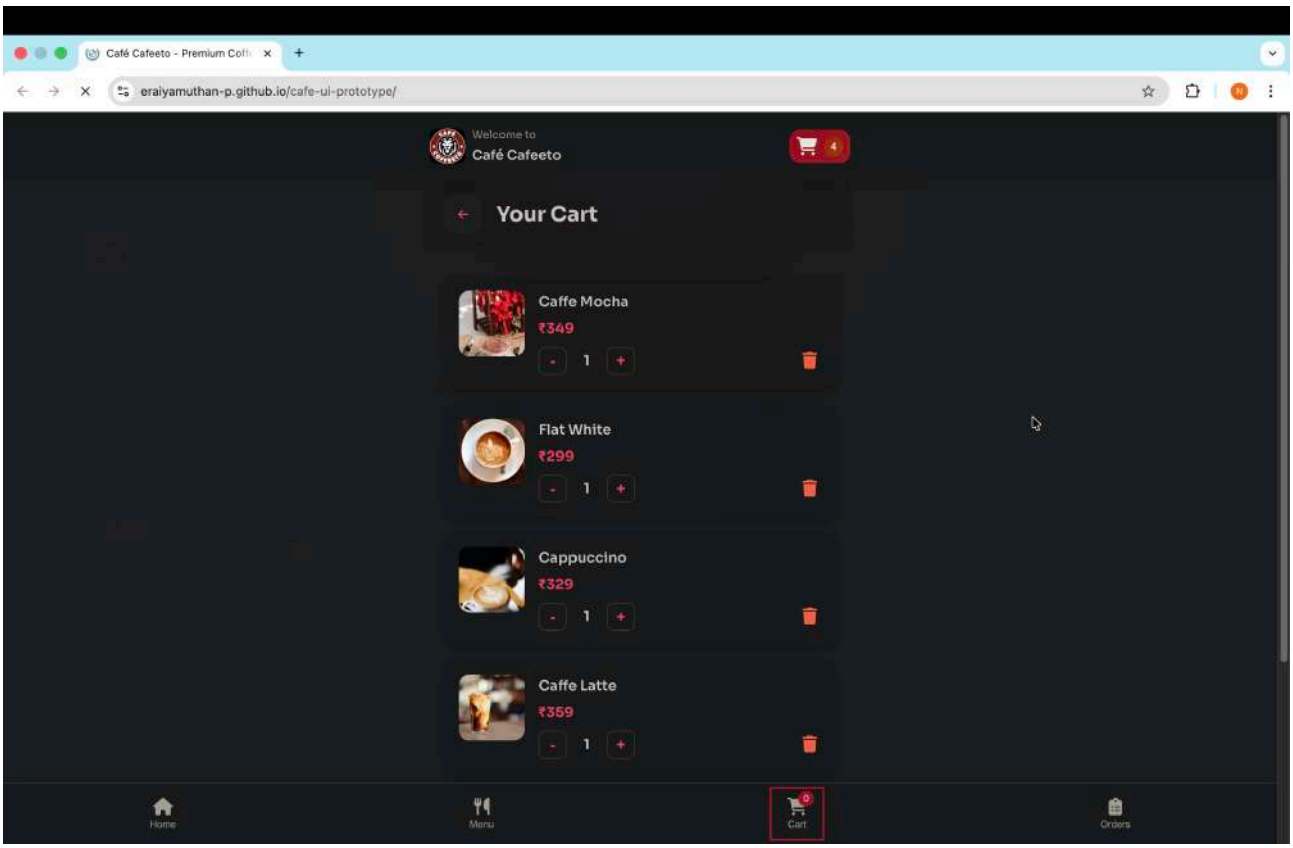
```
import java.time.LocalDateTime;

// Entity
class Delivery {
   private String deliveryId;
   private String deliveryStatus;
   private String deliveryAddress;
   private LocalDateTime deliveryTime;

   public void assignDeliveryPartner() {}
   public void updateDeliveryStatus() {}
   public void getDeliveryDetails() {}
}

// Entity
class OrderDelivery {
   private String orderId;
   private String orderStatus;

   public void getOrderDetails() {}
}

// Boundary Class
class DeliveryPartner {
   private String partnerId;
   private String partnerName;
   private String contactNumber;
```

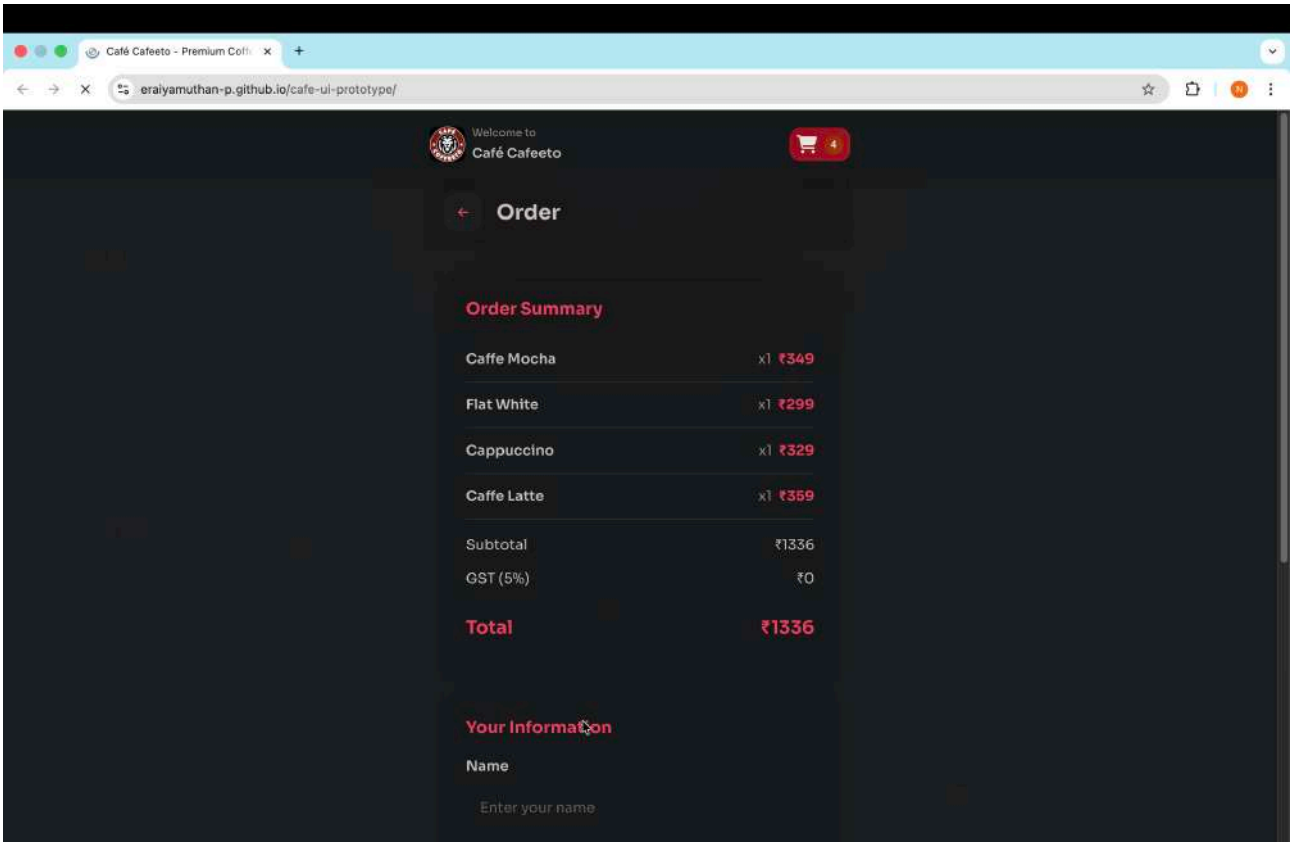
```
    public void acceptDelivery() {}
    public void updateDeliveryProgress() {}
}

// Control Class
class DeliveryService {
    private String serviceId;

    public void scheduleDelivery() {}
    public void notifyCustomer() {}
    public void trackDelivery() {}
}
```

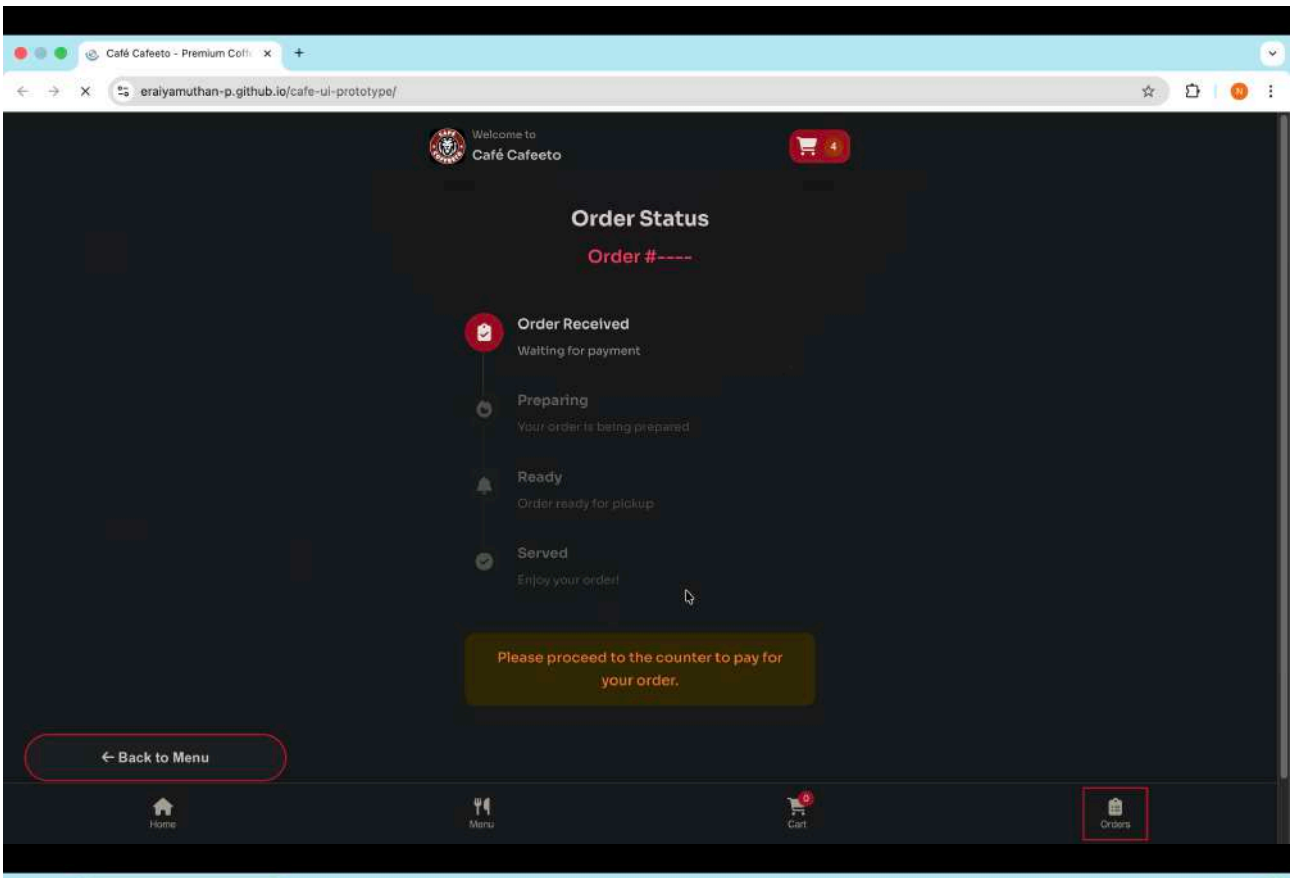
13. Sample UI designs

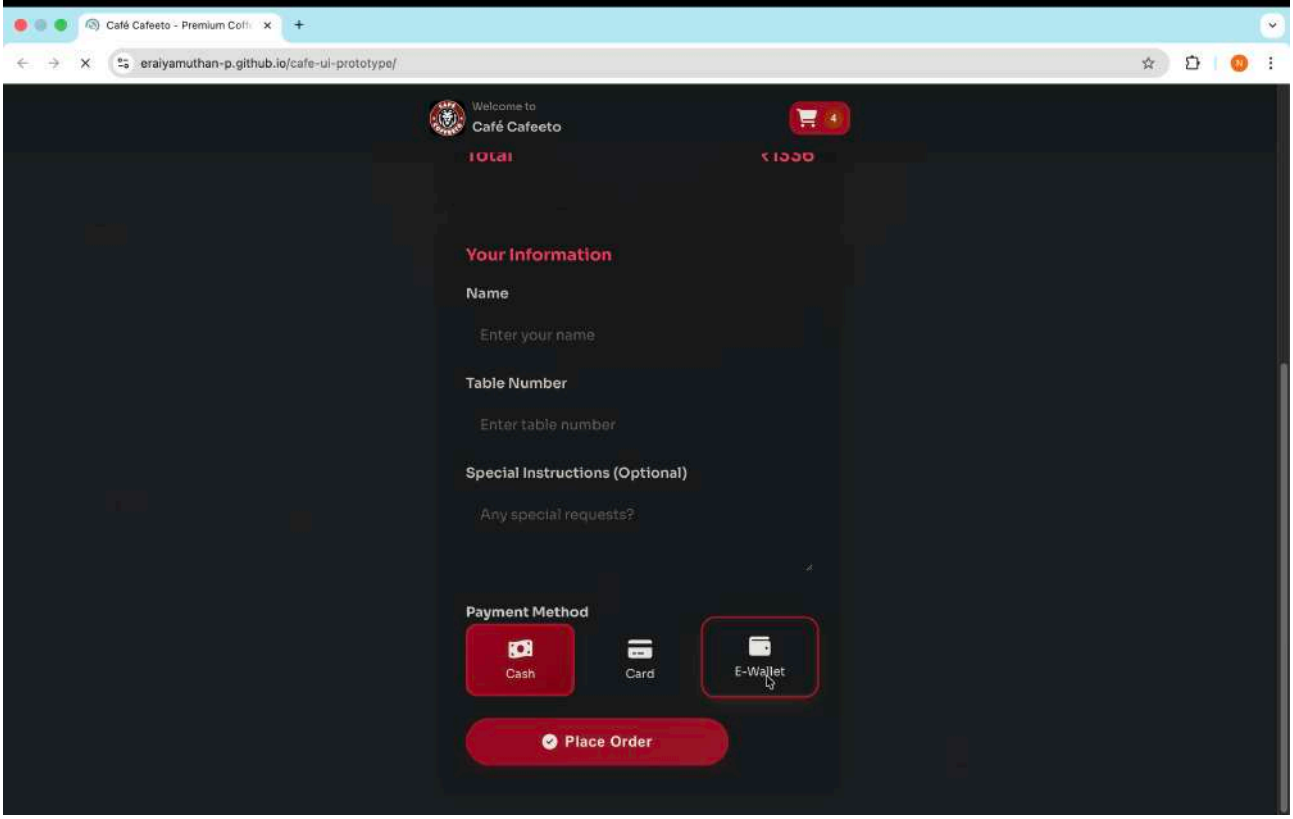





Café Cafeeto - Premium Coff
eraiyamuthan-p.github.io/cafe-ui-prototype/
Welcome to
Café Cafeeto
4
Your Cart
Caffe Mocha
₹349
1
Flat White
₹299
1
Cappuccino
₹329
1
Caffe Latte
₹359
1
Home
Menu
Cart
Orders


Café Cafeeto - Premium Coff
eraiyamuthan-p.github.io/cafe-ui-prototype/
Welcome to
Café Cafeeto
4
Order
Order Summary
Caffe Mocha x1 ₹349
Flat White x1 ₹299
Cappuccino x1 ₹329
Caffe Latte x1 ₹359
Subtotal ₹1336
GST (5%) ₹0
Total ₹1336
Your Information
Name
Enter your name


Café Cafeeto - Premium Coff
eraiyamuthan-p.github.io/cafe-ui-prototype/
Welcome to
Café Cafeeto
4
Order Status
Order #----
Order Received
Waiting for payment
Preparing
Your order is being prepared
Ready
Order ready for pickup
Served
Enjoy your order!
Please proceed to the counter to pay for your order.
← Back to Menu
Home
Menu
Cart
Orders


Café Cafeeto - Premium Coff
eraiyamuthan-p.github.io/cafe-ui-prototype/
Welcome to
Café Cafeeto
4
Your Information
Name
Enter your name
Table Number
Enter table number
Special Instructions (Optional)
Any special requests?
Payment Method
Cash
Card
E-Wallet
Place Order